

RTL Design Sherpa

Bridge Generator Architecture Specification 2.0

October 13, 2026

Table of Contents

1 Bridge Gas Index....17

2 Bridge Hardware Architecture Specification Index....18

2.1 Overview....18

2.2 Related Modules....19

2.3 Navigation....20

2.3.1 Front Matter....20

2.3.2 Chapter 1: Introduction....20

2.3.3 Chapter 2: System Overview....20

2.3.4 Chapter 3: Architecture....20

2.3.5 Chapter 4: Interfaces....20

2.3.6 Chapter 5: Performance....20

2.3.7 Chapter 6: Integration....21

3 Bridge Micro-Architecture Specification Index....22

3.1 Overview....23

3.1.1 Key Features....23

3.1.2 Protocol Conversion Matrix....23

3.1.3 Design Philosophy....24

3.2 Design Notes....25

3.2.1 Version History....25

3.3 References....26

3.3.1 Visual Assets....26

3.3.2 Companion Specifications....26

3.3.3 Project-Level....26
3.3.4 Generator....26
3.4 Navigation....28
3.4.1 Document Organization....28
3.4.2 Quick Navigation....29
4 Bridge: AXI4 Full Crossbar Generator - Product Requirements Document....30
4.1 Executive Summary....31
4.2 Design Philosophy....32
4.2.1 Core Principles....32
4.2.2 Architecture Philosophy....33
4.3 CRITICAL: RTL Regeneration Requirements....35
4.4 1. Product Overview....36
4.4.1 1.1 Purpose....36
4.4.2 1.2 Target Audience....36
4.4.3 1.3 Success Criteria....36
4.4.4 1.4 Implementation Status and Phases....37
4.5 2. Architecture Overview....39
4.5.1 2.1 AXI4 vs AXIS vs APB....39
4.5.2 2.2 Block Diagram....40
4.5.3 2.3 Key Components....41
4.6 3. Functional Requirements....42
4.6.1 3.1 AXI4 Protocol Compliance....42
4.6.2 3.2 Address Decoding....42
4.6.3 3.3 Arbitration Strategy....43

4.6.4 3.4 Burst Handling....43
4.7 4. Non-Functional Requirements....44
4.7.1 4.1 Performance....44
4.7.2 4.2 Resource Usage (Measured)....44
4.7.3 4.3 Quality Requirements....45
4.8 5. Interface Specifications....46
4.8.1 5.1 AXI4 Master Interfaces (M × 5 channels)....46
4.8.2 5.2 AXI4 Slave Interfaces (S × 5 channels)....47
4.8.3 5.3 Configuration Is Generation-Time, Not Parameters....47
4.9 6. Performance Modeling....49
4.9.1 6.1 Analytical Model....49
4.9.2 6.2 Resource Scaling....49
4.9.3 6.3 Comparison with Other Crossbars....50
4.10 7. Generator Architecture....51
4.10.1 7.1 Python Generator Structure....51
4.10.2 7.2 Address Map Configuration....52
4.11 8. Comparison with APB and Delta....54
4.11.1 8.1 Code Reuse from APB Generator....54
4.11.2 8.2 Code Reuse from Delta Generator....54
4.11.3 8.3 Complexity Comparison....54
4.12 9. Use Cases....56
4.12.1 9.1 Multi-Core Processor Interconnect....56
4.12.2 9.2 DMA + Accelerator System....56
4.12.3 9.3 FPGA System Integration....56

4.13 10. Testing Strategy....57
4.13.1 10.1 FUB (Functional Unit Block) Tests....57
4.13.2 10.2 Integration Tests....57
4.13.3 10.3 Performance Validation....57
4.14 11. Documentation Plan....58
4.14.1 11.1 Specifications (Before Code)....58
4.14.2 11.2 Performance Analysis (Before Implementation)....58
4.14.3 11.3 Code Generation....58
4.14.4 11.4 Verification....58
4.15 12. Success Metrics....59
4.15.1 12.1 Functional Completeness....59
4.15.2 12.2 Performance Targets....59
4.15.3 12.3 Educational Value....59
4.15.4 12.4 Reusability....59
4.16 12.2 PDF Generation Location....60
4.17 13. Future Enhancements....61
4.17.1 13.1 Short-Term (Post-Initial Release)....61
4.17.2 13.2 Long-Term....61
4.18 14. Risk Assessment....62
4.19 15. Project Timeline (Estimated)....63
4.20 16. Glossary....64
4.21 References....65
5 Claude Code Guide: Bridge Subsystem....67
5.1 CRITICAL: Read Architecture Documents First....68

5.2 Quick Context..... 69
5.3 Target Architecture: Intelligent Width-Aware Routing.....70
5.3.1 Efficient Multi-Width Design.....70
5.3.2 Why This Architecture.....70
5.3.3 Per-Master Output Paths.....70
5.3.4 Benefits..... 71
5.3.5 Implementation Status..... 71
5.4 CRITICAL RULE #0: RTL Regeneration Requirements.....72
5.5 Critical Pitfalls: slave_select_* Reversion After Handshake.....73
5.5.1 The Problem: Combinational Address Decode.....73
5.5.2 Visible Symptoms..... 73
5.5.3 The Solution (MANDATORY in All Generators).....73
5.5.4 When This Bug Appears.....74
5.6 MANDATORY: Project Organization Pattern.....75
5.6.1 Required Directory Structure..... 75
5.6.2 Testbench Class Location (MANDATORY).....76
5.6.3 Test File Pattern (MANDATORY).....76
5.7 Critical Rules for This Subsystem.....77
5.7.1 Rule #0.1: Testbench Architecture - MANDATORY SEPARATION.....77
5.7.2 Rule #1: TBBase and the three mandatory methods.....77
5.7.3 Rule #3: Use GAXI Components for Protocol Handling.....77
5.7.4 Rule #4: Queue-Based Verification.....78
5.8 TOML/CSV-Based Bridge Generator (Phase 2 Complete).....79
5.8.1 Overview..... 79

5.8.2 Quick Start....79
5.8.3 Configuration Format Details....80
5.8.4 Channel-Specific Masters (Phase 2 Feature)....81
5.8.5 Example: RAPIDS-Style Configuration....81
5.8.6 Common User Questions....82
5.8.7 Generator Output Structure....85
5.8.8 Testing Generated Bridges....85
5.9 Bridge Architecture Quick Reference....87
5.9.1 Generated Bridge Crossbar Structure....87
5.9.2 Key Features....87
5.9.3 Address Map....87
5.10 Test Organization....89
5.10.1 Test Levels....89
5.11 Common User Questions and Responses....90
5.11.1 Q: “How does the Bridge work?”....90
5.11.2 Q: “How do I generate a Bridge?”....91
5.11.3 Q: “How do I test a Bridge?”....91
5.12 PDF Generation Location....92
6 Document Information....93
6.1 Bridge Generator Architecture Specification....93
6.2 Revision History....94
6.3 Document Purpose....95
6.4 Intended Audience....96
6.5 Related Documents....97

7 What the Generator Produces....98
7.1 Generated Artifact Family....99
7.2 What Each Variant Means....100
7.3 Scope of Generation....101
8 When to Regenerate....102
8.1 Triggers for Full Regeneration....103
8.2 Regeneration Workflow....104
8.3 When Inputs (Not Code) Change....105
8.4 Identifying Generated Files....106
9 Generator Entry Points....107
9.1 Single-Bridge Mode....108
9.2 Bulk Mode....109
9.3 Makefile Targets....110
9.4 Return Value....111
10 Bridge-Level TOML Keys....112
10.1 [bridge] scalar keys....113
10.2 [bridge.clock] table....114
10.3 Crossbar and arbitration keys....115
10.4 Monitor and cfg subsystem keys....116
10.5 [bridge.mon_group] table....118
10.6 [bridge.defaults] table....119
11 Port Specification Schema....120
11.1 Common port fields....121
11.2 Slave-only fields....123

11.3 Optional behavior fields....124
11.4 axi5_features values....126
11.5 Channel semantics....127
11.6 Example....128
12 Interface Wrapper Configuration....129
12.1 interface.type values....130
12.2 interface.skid_depths table....131
12.3 How the wrapper selection flows....132
12.4 Protocol-specific wrapper notes....133
13 Monitor and CFG Subsystem Keys....134
13.1 Monitor subsystem....135
13.1.1 Bridge-level monitor keys....135
13.1.2 Per-port monitor keys....135
13.1.3 [bridge.mon_group] table....136
13.2 CFG register block....137
13.2.1 What changes....137
13.2.2 Per-port cfg fields....137
14 Validation Rules....139
14.1 Field validation (validate_required_fields)....140
14.2 Channel and protocol validation....141
14.3 Protocol-specific constraints....142
14.3.1 APB / APB5 (validate_apb_constraints)....142
14.3.2 Wishbone B4 (validate_wb4_constraints)....142
14.3.3 AXI3 (validate_axi3)....142

14.3.4 AXI5 (validate_axi5)....142
14.4 Master-slave compatibility (validate_master_slave_compatibility)....143
14.5 Address map validation (validate_address_map)....144
14.6 Slave ID width (validate_slave_id_widths)....145
14.7 CDC constraints (validate_cdc_constraints)....146
14.8 Warning for dropped sideband (warn_axi5_dropped_sideband)....147
15 Pipeline Overview....148
15.1 Pipeline diagram....149
15.1.1 Figure 3.1: Bridge Generator Pipeline....149
15.2 Key files by stage....150
16 Configuration Loading....152
16.1 TOML parsing....153
16.2 Connectivity CSV....154
16.3 BridgeConfig construction....155
16.4 Normalization....156
17 Validation....157
17.1 Validation order....158
17.2 Error behavior....159
17.3 Examples of validation failures....160
17.4 Adding a new validation rule....161
18 Adapter and Crossbar Emission....162
18.1 Translation to emitter structs....163
18.2 BridgeModuleGenerator construction....164
18.3 Sub-generators....165

18.4 Master adapter (AdapterGenerator)....166
18.5 Slave adapter (SlaveAdapterGenerator)....167
18.6 Crossbar (CrossbarGenerator)....168
19 Filelist and Test Generation....169
19.1 Filelist generation....170
19.1.1 Filelist sections....170
19.2 Test generation....171
20 Directory Layout....172
20.1 Variant directories....173
21 Package File....174
21.1 What it contains....175
21.2 Excerpt from a generated package....176
21.3 Why it matters....177
22 Top Module....178
22.1 Module signature....179
22.2 Internal structure....180
22.3 Excerpt: subtractive catch-all instantiation....181
22.4 Excerpt: adapter wiring pattern....182
23 Adapters and Crossbar....183
23.1 Master adapter (<name>_adapter.sv)....183
23.1.1 Per-width paths....183
23.2 Slave adapter (<name>_adapter.sv)....184
23.2.1 Response routing....184
23.3 Crossbar (<bridge_name>_xbar.sv)....185

24 Filelist....186
24.1 Path convention....187
24.2 Example filelist head....188
24.3 Filelist is generated, not hand-maintained....189
25 Generated Tests....190
25.1 TB class....191
25.2 Test runner....192
25.3 Monitor stress test....193
25.4 Regeneration discipline for tests....194
26 Full Regeneration Rule....195
26.1 Why partial regeneration is dangerous....196
26.2 Required workflow....197
26.3 Code changes that trigger full regeneration....198
26.4 Files that are never regenerated....199
26.5 Version-mismatch symptoms....200
27 Generated Examples....201
27.1 Representative fixtures....202
27.2 How to verify a regeneration....203
27.3 Fixture as documentation....204
28 DV Suite....205
28.1 Test structure....206
28.2 Running tests....207
28.3 Test levels....208
28.4 Verification coverage....209

28.5 When generator changes affect DV.....210
29 Lint and Synthesis Checks.....211
29.1 Verilator lint.....212
29.2 Out-of-context synthesis.....213
29.3 When to run checks.....214
30 Adding a Protocol.....215
30.1 Step 1: Add the protocol to the validator.....216
30.2 Step 2: Add converter/wrapper selection.....217
30.3 Step 3: Update the filelist.....218
30.4 Step 4: Add a representative fixture.....219
30.5 Step 5: Run the full regeneration and test.....220
30.6 Example: how APB was added.....221
31 Adding a Bridge-Level Option.....222
31.1 Step 1: Add the dataclass field.....223
31.2 Step 2: Add the TOML loader mapping.....224
31.3 Step 3: Validate if necessary.....225
31.4 Step 4: Thread it into the emitter.....226
31.5 Step 5: Use the option in BridgeModuleGenerator.....227
31.6 Step 6: Document and test.....228
31.7 Example: xbar_pipeline.....229
32 Adding a Port-Level Option.....230
32.1 Step 1: Add the PortSpec field.....231
32.2 Step 2: Load it from TOML.....232
32.3 Step 3: Validate it.....233

32.4 Step 4: Thread it into MasterConfig / SlaveInfo....234
32.5 Step 5: Use it in the adapter generator....235
32.6 Step 6: Update the filelist if needed....236
32.7 Example: cdc....237
33 Testing an Extension....238
33.1 Minimum test plan....239
33.2 Regression fixtures....240
33.3 Documentation....241

List of Figures

Figure 3.1: Bridge Generator Pipeline.....148

List of Tables

No tables in this document.

1 Bridge Gas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Bridge Hardware Architecture Specification Index

2.1 Overview

Version: 2.0 **Date:** 2026-10-12 **Purpose:** High-level hardware architecture specification for Bridge component

---

2.2 Related Modules

- **Bridge MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

2.3 Navigation

Note: Every chapter below is one source file; the document build assembles the spec from these links.

2.3.1 Front Matter

- Document Information

2.3.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

2.3.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

2.3.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Protocol Support

2.3.5 Chapter 4: Interfaces

- AXI4 Interface Overview
- APB Interface Overview
- Clock and Reset
- AXI5 and APB5 Interfaces (AMBA5)
- Unmapped-Address Handling
- Wishbone B4 Interface

2.3.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resources

2.3.7 Chapter 6: Integration

- System Requirements

- Parameter Configuration
- Verification Strategy
- Synthesis and Implementation
- AXI3 Integration

Last Updated: 2026-10-11 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Bridge Micro-Architecture Specification Index

Component: Bridge (Multi-Protocol Crossbar Generator) **Version:** 2.0 **Date:** 2026-10-12 **Purpose:** Detailed micro-architecture specification for Bridge component

---

3.1 Overview

Bridge is a TOML-driven generator that produces AXI4 crossbars with automatic width conversion, protocol conversion (AXI3, AXI4, AXI5, AXI4-Lite, AXI5-Lite, APB, APB5, Wishbone B4 — per-port, on slave or master sides; ACE coherency is NOT supported), and channel-specific masters (read-only, write-only, or full). This is the internals document — the one you open when you need to know what the generated RTL actually does, block by block, rather than what the brochure says.

3.1.1 Key Features

- **Multi-Protocol Support:** AXI4, AXI3, AXI5, AXI4-Lite, AXI5-Lite, APB, APB5, and Wishbone B4 conversion, on slave or master ports (ACE coherency NOT supported)
- **CSV-Driven Configuration:** Human-readable port and connectivity definitions
- **Channel-Specific Masters:** Write-only (wr), read-only (rd), or full (rw) support
- **Automatic Width Conversion:** Upsize/downsize for data width mismatches
- **Out-of-Order Support:** NONE. Responses route by in-order bridge_id FIFO position; a slave that reorders between IDs misroutes (bridge BUG-008, was BRIDGE-010)
- **Custom Signal Prefixes:** Unique prefixes per port for clean integration

That out-of-order bullet is the one people skim past and regret. The bridge does not sort responses back into order — it trusts the FIFO position, so the trust has to be mutual.

3.1.2 Protocol Conversion Matrix

What happens to a transaction depends on the master/slave protocol pairing:

Master Protocol	Slave Protocol	Conversion
AXI4	AXI4	Direct or width convert
AXI4	AXI3	`axi4_to_axi3` protocol convert; same data width
AXI3	AXI4	`axi3_to_axi4` protocol convert; same data width

Master Protocol	Slave Protocol	Conversion
AXI4	AXI4-Lite	Protocol downgrade
AXI4	APB	Full protocol conversion

3.1.3 Design Philosophy

Efficient Multi-Width Routing: - Direct connections where widths match (zero conversion overhead) - Width converters only where needed - No fixed internal crossbar width

Resource Optimization: - Channel-specific masters reduce port count by 40-60% - Per-path width converters instead of global conversion - Minimal logic for matching-width connections

3.2 Design Notes

3.2.1 Version History

Version 1.2 (2026-09-07): qc round_1/round_2 correctness pass. Retired the ID-extension architecture from every page that claimed it (IDs are pass-through; responses route by an in-order `bridge_id` FIFO – see bridge BUG-008 (was BRIDGE-010) for the ordering requirement that creates). Corrected the out-of-range responder against `axi4_subtractive_slave` (RLAST on the final beat only, DECERR, 0xDEADBEEF), the monbus group instantiation, the transaction timeout that does not exist, and the strobe-mapping example. Documented the subtractive catch-all and its status/IRQ path.

Version 1.1 (2026-06-04): content sync to RTL state at that date.

Version 1.0 (2026-01-03): - Initial specification release - Restructured from single spec to HAS/MAS format - Complete block-level documentation - FSM and ID management details

3.3 References

3.3.1 Visual Assets

Diagram sources and their renders live in two directories:

- **Source Files:**
  - `assets/graphviz/*.gv` - Graphviz source diagrams
  - `assets/puml/*.puml` - PlantUML FSM diagrams
- **Rendered Files:**
  - `assets/graphviz/*.png` - Rendered block diagrams
  - `assets/puml/*.png` - Rendered FSM diagrams

3.3.1.1 Architecture Diagrams

1. **Master Adapter** - assets/graphviz/master_adapter.png
2. **Slave Router** - assets/graphviz/slave_router.png
3. **Crossbar Core** - assets/graphviz/crossbar_core.png
4. **ID Management** - assets/graphviz/id_management.png
5. **Width Conversion** - assets/graphviz/width_conversion.png
6. **Protocol Conversion** - assets/graphviz/protocol_conversion_apb.png
7. **Response Routing** - assets/graphviz/response_routing.png
8. **Error Handling** - assets/graphviz/error_handling.png

3.3.1.2 FSM Diagrams

1. **AW Arbiter FSM** - assets/puml/aw_arbiter_fsm.png
2. **AR Arbiter FSM** - assets/puml/ar_arbiter_fsm.png

3.3.2 Companion Specifications

- **Bridge HAS** - Hardware Architecture Specification (high-level)

3.3.3 Project-Level

- **PRD.md:** ../../PRD.md - Complete product requirements document
- **CLAUDE.md:** ../../CLAUDE.md - AI assistant integration guide

3.3.4 Generator

- **Generator:** `../../bin/bridge_generator.py` - CSV/TOML-based generator script (with `../../bin/bridge_pkg/`)

- **Test Configs:** `../../bin/test_configs/` - Example TOML/CSV configurations

3.4 Navigation

3.4.1 Document Organization

Each chapter below is one source file.

3.4.1.1 Front Matter

- Document Information

3.4.1.2 Chapter 1: Introduction

- Overview

3.4.1.3 Chapter 2: Block Descriptions

- Master Adapter
- Slave Router
- Crossbar Core
- Arbitration
- ID Management
- Width Conversion
- Protocol Conversion
- Response Routing
- Error Handling
- AMBA5 Boundary and Native Sideband

3.4.1.4 Chapter 3: FSM Design

- Arbiter FSMs
- Transaction Tracking

3.4.1.5 Chapter 4: ID Management

- CAM Architecture – describes a design that was NEVER BUILT; see ID Tracking for the mechanism actually generated
- ID Tracking Tables

3.4.1.6 Chapter 5: Converters

- Width Converters
- APB Converters

3.4.1.7 Chapter 6: Generated RTL

- Module Structure

- Signal Naming

3.4.1.8 Chapter 7: Verification

- Test Strategy
- Debug Guide

3.4.2 Quick Navigation

3.4.2.1 For New Users

1. Start with Overview for a tour of what Bridge can do
2. Read Block Diagram for the architecture
3. Study Arbitration for operational details
4. Reference Module Structure for generated RTL

3.4.2.2 For Integration

- **Protocol support:** See Protocol Conversion for AXI4/AXI3/APB/AXI-Lite
- **Width handling:** See Width Conversion for data width mismatches
- **ID management:** See ID Management for transaction tracking
- **Error handling:** See Error Handling for OOR and timeout

Last Updated: 2026-10-11 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Bridge: AXI4 Full Crossbar Generator - Product Requirements Document

Project: Bridge **Version:** 2.2 **Status:** Phase 2 Complete - Simplified Architecture with Hard Limits
Created: 2025-10-18 **Last Updated:** 2026-09-13

4.1 Executive Summary

Bridge is a Python-based AXI4 crossbar generator that produces simple, performant SystemVerilog RTL for connecting multiple AXI4 masters to multiple AXI4 slaves. The name follows the infrastructure theme - bridges connect different regions, enabling communication across divides, just like crossbars connect masters and slaves.

Design Philosophy: A simple AMBA fabric that is performant, but makes no attempt to support all features. ID and address widths are PER PORT, set in the TOML (`id_width`, `addr_width`) – see §5.3, “Configuration Is Generation-Time, Not Parameters”. An earlier revision of this line claimed hard 8-bit/64-bit limits; the generator has never enforced them, and the shipped configs use 2-, 4- and 8-bit IDs.

Key Differentiator from Delta: - **Delta:** AXI-Stream crossbar (streaming data, single channel, simple routing) - **Bridge:** AXI4 full crossbar (memory-mapped, 5 channels, burst support, `bridge_id` sideband routing)

Key Differentiator from Commercial IP: - **Commercial AXI4 Crossbars:** Feature-complete, support every AXI4 edge case, complex configurability - **Bridge:** Simple and performant, supports common use cases, hard limits for simplicity

Target Use Case: High-performance memory-mapped interconnects for multi-core processors, accelerators, and memory controllers where simplicity and performance matter more than spec compliance.

4.2 Design Philosophy

Vision: A simple AMBA fabric that is performant, but makes no attempt to support all features.

4.2.1 Core Principles

1. Simplicity Over Completeness - Focus on common use cases, not edge cases - Implement what's needed for real hardware, not AXI4 spec compliance theater - Prefer straightforward implementations over complex feature sets

2. Performance First - Direct connections where possible (matching widths = zero overhead) - Minimal conversion logic in critical paths - Optimize for throughput and latency, not feature count

3. Hard Limits for Simplicity

We enforce uniform widths on certain parameters to eliminate complexity:

Parameter	Hard Limit	Rationale
ID Width	Per port, set at generation	NOT fixed at 8. The TOML sets `id_width` per port and the generated bridge bakes it in – `bridge_2x2_rw` has 4-bit IDs. This row previously said "8 bits (fixed)", which the generator has never enforced.
Address Width	Per port, set at generation	NOT fixed at 64. `bridge_2x2_rw` is a 32-bit-address bridge. Same correction as the row above.
Data Width	Variable (32b-512b)	Width converters ONLY for data. Commonly needed for bandwidth optimization.

Why This Works: - ID width conversion adds complexity with minimal benefit (nobody needs >256 outstanding transactions per master) - Address width conversion is rarely needed (peripheral addresses fit in 32-bit, but using 64-bit everywhere is simpler) - Data width conversion is the ONLY width conversion that provides real value (bandwidth matching)

4. What We DON'T Support (By Design)

Features intentionally excluded for simplicity: - (Both of these WERE excluded once and no longer are: ID width and address width are per-port generation inputs. Left here as history because the "design differentiator" argument above still cites them.) - ~~No AXI4-Lite protocol variant~~ – `axil` and `axil5` ARE supported, as slave protocols (converted at the boundary by `axi4_to_axil4_{rd,wr}` / `axi4_to_axil5_{rd,wr}`) and, since bridge TASK-004 (was BRIDGE-014), as master protocols (tied-off promotion plus the AXI5-Lite sideband surface). `apb` / `apb5` likewise work on both sides (`axi4_to_apb{4,5}_shim` at a slave, `apb{4,5}_to_axi4` at a master), Wishbone B4 since bridge TASK-008 (was BRIDGE-019) (`axi4_to_wb4` / `wb4_to_axi4`), and AXI3 (`axi4_to_axi3` at a slave, `axi3_to_axi4` at a master). - No ACE protocol extensions (cache coherency) - ~~No AXI5 features~~ – interop-mode AXI5 sideband, native sideband through the structs, atomics of every class and APB5/AXI5-Lite ports were delivered by bridge TASK-002 (was BRIDGE-002); what is NOT built is a fabric that is itself AXI5 (bridge TASK-007, was BRIDGE-018) - QoS, Region and REQUEST-side User (AWUSER/ARUSER/WUSER) are routed since `c2955863`/`2b229516`. RESPONSE-side User is not: `BUSER`/`RUSER` exist as ports but the master adapters tie them to zero (`.fub_axi_buser(1'b0)`, `.fub_axi_ruser(1'b0)`), so a slave's response User never reaches its master. An earlier correction of this line said "end to end", which was true only of the request side.

5. What We DO Support

Core AXI4 features that matter: - Full 5-channel AXI4 protocol (AW, W, B, AR, R) - Burst transactions (INCR, WRAP, FIXED) - In-order completion. Responses are routed by a per-master `bridge_id` sideband and an in-order FIFO, NOT by matching AXI IDs – see FR-2. - Multiple outstanding transactions per master, **to one slave at a time**. The generated master adapters carry a single-outstanding-target gate (`aw_gate_ok` / `ar_gate_ok`): a new AW/AR to a DIFFERENT slave is held off until the outstanding ones drain. - Channel-specific masters (write-only, read-only, read-write) - Data width conversion (32b ↔️ 64b ↔️ 128b ↔️ 256b ↔️ 512b) - Configurable M×N topology (1-32 masters, 1-256 slaves) - Fair round-robin arbitration per slave

4.2.2 Architecture Philosophy

Delivered: Intelligent width-aware routing, not a fixed-width crossbar. The generated adapters carry per-width paths and convert only at a mismatch.

OLD Approach (What We're Moving Away From):

```
Master (64b) → Upsize(64→256) → Fixed 256b Crossbar → Downsize(256→64)
→ Slave (64b)
```

- Two conversions for same-width connections (wasteful!)
- Artificial bandwidth bottleneck
- Unnecessary logic and latency

NEW Approach (Target Architecture):

```
Master (64b) → Direct Connection → Slave (64b)    [0 conversions!]
Master (512b) → Conv(512→64) → Slave (64b)         [1 conversion]
```

- Per-master paths to each unique slave width it connects to
- Direct paths where widths match (zero overhead)
- Converters only where actually needed
- Router selects appropriate path based on address decode

Result: Minimal conversion logic, maximum performance, pragmatic simplicity.

4.3 CRITICAL: RTL Regeneration Requirements

ALL generated RTL files MUST be deleted and regenerated together whenever ANY generator code changes.

Why This Matters: - Generated RTL files may have interdependencies (bridges, wrappers, integrators) - Generator code changes can create version mismatches between files - Partial regeneration creates subtle incompatibilities that cause test failures - Even “small innocuous” generator changes can have cascading effects

The Rule:

```
# WRONG - Partial regeneration
./bridge_generator.py --masters 5 --slaves 3 --output ../rtl/
# Only regenerates bridge_axi4_flat_5x3.sv
# Other files (wrappers, integrators) now mismatched!

# CORRECT - Full regeneration
cd bin
make clean                                  # Delete ALL generated
bridges (rtl/generated/)
make all                                    # Regenerate everything from
bridge_batch.csv
```

Generator Files That Trigger Full Regeneration: - `bin/bridge_generator.py` - Main bridge generator (CSV/TOML driven) - Any Python file in `bin/bridge_pkg/` (components/, generators/, config, csv_parser, ...) - Any Jinja template in `bin/bridge_pkg/jinja_templates/` - `bridge_wrapper_generator.py` - Wrapper generation - **Any** Python file in `projects/components/fabric-gen-ip/bridge/bin/`

Symptoms of Version Mismatch: - Tests that previously passed now fail - Simulation errors about missing signals - Mismatched port widths or counts - Address decode routing to wrong slaves

Think of Generated RTL Like Compiled Code: When you update a compiler, you don’t selectively recompile - you rebuild everything. When you update a generator, you don’t selectively regenerate - you regenerate everything.

4.4 1. Product Overview

4.4.1 1.1 Purpose

Bridge automates generation of AXI4 crossbar infrastructure: - **Python code generation** - Parameterized RTL generation (similar to APB/Delta) - **Performance modeling** - Analytical + simulation validation - **Flat topology** - Full M×N interconnect matrix - **`bridge_id` sideband routing** - in-order response return (see FR-2) - **Burst optimization** - Pipelined burst transfers

4.4.2 1.2 Target Audience

Primary Users: - RTL designers building SoC interconnect - System architects evaluating interconnect topologies - Verification engineers needing crossbar testbenches - Students learning AXI4 protocol and interconnects

Educational Focus: - Demonstrates AXI4 protocol complexity - Shows arbitration strategies for memory-mapped busses - Teaches ID-based transaction tracking - Illustrates burst optimization techniques

4.4.3 1.3 Success Criteria

Functional: - [x] Generates working AXI4 crossbar RTL (bridge_generator.py) - [x] Generates CSV/TOML-configured bridges (bridge_generator.py + bridge_pkg/; the earlier separate bridge_csv_generator.py was merged in) - [x] Passes Verilator lint - [x] Supports 1-32 masters, 1-256 slaves - [ ] Handles out-of-order completion via IDs. NOT IMPLEMENTED: `bridge_cam.sv` exists but is instantiated by no generated module. The slave adapters run "Bridge ID Tracking - FIFO Mode (In-Order)" and 93 generated files still declare `cam_wr_allocate`/`cam_rd_allocate` without driving them. - [x] Supports burst lengths 1-256 beats - [x] Channel-specific masters (wr/rd/rw) for resource optimization (Phase 2) - [x] APB converter integration – DELIVERED. `axi4_to_apb4_shim` is instantiated by `apb_periph_adapter.sv` in every mixed config, and `bridge_1x2_rw_apb5`, `bridge_1x2_rw_apb5_mon` and the four `mix_*` bridges ship with APB slaves. - [x] AXI3 converter integration – DELIVERED. `axi4_to_axi3` and `axi3_to_axi4` boundary converters are emitted for `protocol = "axi3"` slave and master ports; `bridge_1x2_axi3`, `bridge_2x2_axi3_masters`, `bridge_2x2_axi3_slaves` and `bridge_mix_e` ship as validation fixtures.

Performance: - [ ] Latency ≤ 3 cycles for single-beat transactions. **Unverified for the generated bridge.** The generated path crosses four skid buffers on the round trip (adapter timing wrappers on both sides), and gaxi_skid_buffer registers its output side, so an ideal zero-wait slave still costs >=4 cycles of bridge-added latency before arbitration. The 2-3 cycle figure in 6.1 describes the wrapperless flat crossbar, not what the generator emits. - [ ] Throughput: M×N paths do NOT all transfer concurrently. Each master is held to ONE target slave at a time (`aw_gate_ok`/`ar_gate_ok`, "single-outstanding-target"), a deadlock fix, not an oversight. Concurrency is across MASTERS, not across a master's targets. - [x] Performance models implemented (bridge_model.py - V1 Flat) - [ ] Fmax ≥ 300 MHz on UltraScale+ FPGAs (pending synthesis validation)

Quality: - [x] Complete specifications (PRD) before code - [x] Performance models validate requirements (bridge_model.py) - [x] All generated RTL Verilator verified - [x] Integration examples provided (CSV examples) - [x] Comprehensive documentation (bin/GENERATOR_ARCHITECTURE.md, docs/bridge_has/, docs/bridge_mas/)

4.4.4 1.4 Implementation Status and Phases

Unified Generator (bridge_generator.py):

The bridge generator now supports both TOML/CSV configuration and legacy array-indexed modes:

Configuration Modes: 1. **TOML/CSV Mode (Preferred)** - TOML port configuration (`bridge_name.toml`) - CSV connectivity matrix (`bridge_name_connectivity.csv`) - Custom port prefixes (`rapids_m_axi_`, `cpu_m_axi_`, etc.) - Interface wrapper integration (timing isolation) - Mixed protocols (AXI4 + AXI3 + APB + AXI4-Lite slaves) - Channel-specific masters (wr/rd/rw) - Status: Phase 2 complete, Phase 3 pending

2. **Legacy CSV Mode (Backwards Compatible)**
   - Separate `ports.csv` and `connectivity.csv` files
   - Migration path to TOML format
   - See `bin/test_configs/README.md` for conversion guide

Phase Status:

Phase 1: CSV Configuration COMPLETE - CSV parser (ports.csv, connectivity.csv) - Port generation with custom prefixes - Converter identification logic - Basic crossbar instantiation

Phase 2: Channel-Specific Masters COMPLETE (2025-10-26) - Added `channels` field to PortSpec (rw/wr/rd) - Conditional port generation based on channels - **Resource Optimization:** - wr (write-only): AW, W, B channels only → 39% port reduction - rd (read-only): AR, R channels only → 61% port reduction - rw (full): All 5 channels - Width converter awareness (only generate needed converters) - Example: 4-master bridge saves 35% signals with optimized channels

Phase 3: APB Converter Integration – DELIVERED - AXI2APB converter module: `axi4_to_apb4_shim`, emitted by `axi4_to_apb4_shim_component.py` - APB signal packing/unpacking: in the shim - APB converter instantiation in generated bridges: `apb_periph_adapter.sv` and the `periph5_adapter.sv` variants - Width converter + APB converter chaining: the `*_mon` builds chain the AXI4 timing wrapper into the shim - End-to-end testing with APB slaves: `test_bridge_1x2_rw_apb5*` and the `mix_*` suites, all in the 72-test FULL regression - The “placeholders with TODO comments” status was stale. This block said PENDING while the shim it describes was shipping in six configurations.

Additional Resources: - `models/bridge_model/bridge_model.py` - Performance modeling (V1 Flat implemented) - `rtl/bridge_cam.sv` - CAM for ID tracking. **Not instantiated by any generated module**; kept for the OOO work FR-2 describes as not implemented. - See `bin/GENERATOR_ARCHITECTURE.md` for the generator/build mechanics - See `docs/bridge_has/`

and `docs/bridge_mas/` for architecture diagrams and specs (the older BRIDGE_CURRENT_STATE.md / BRIDGE_ARCHITECTURE_DIAGRAMS.md were superseded)

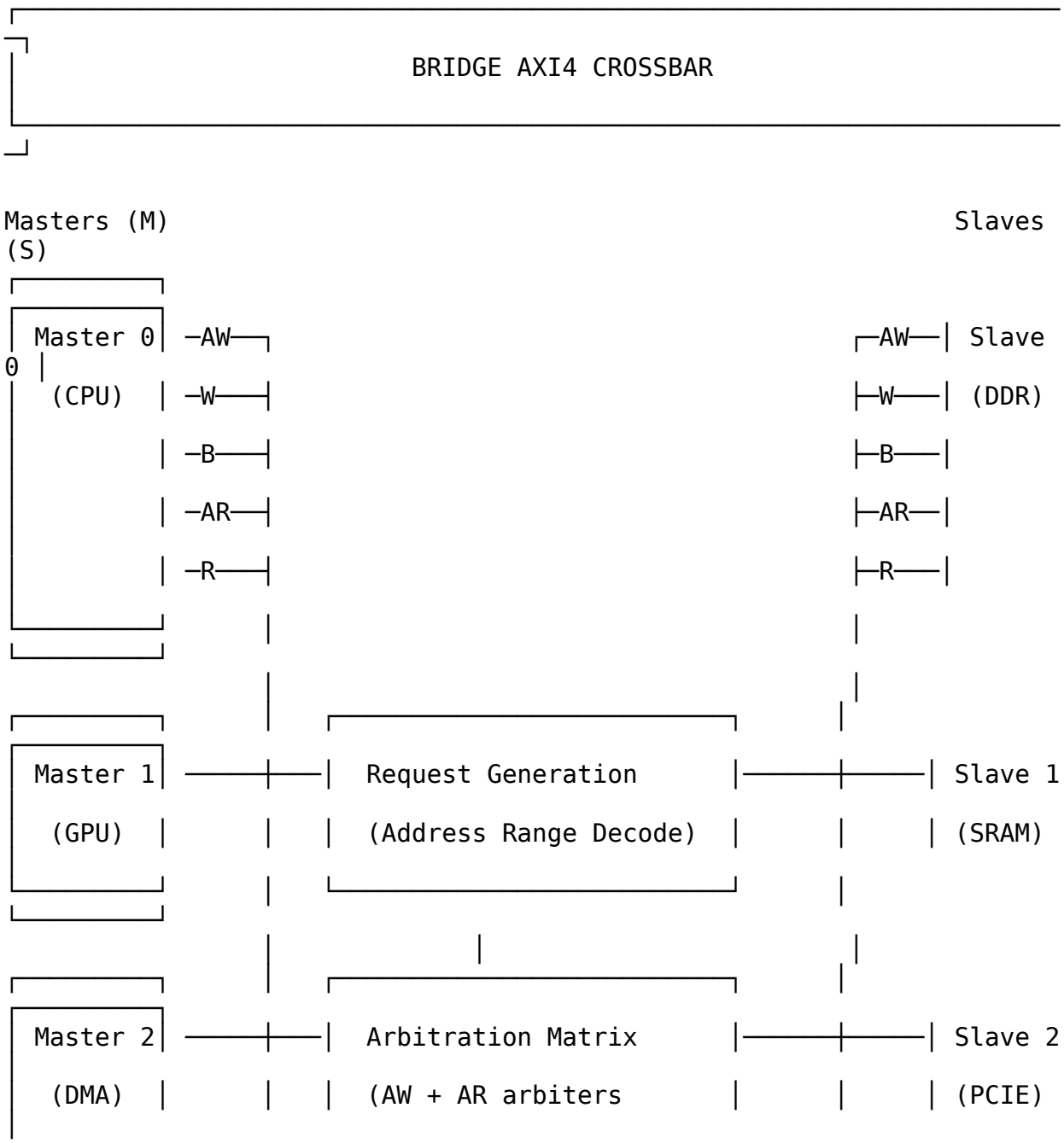
4.5 2. Architecture Overview

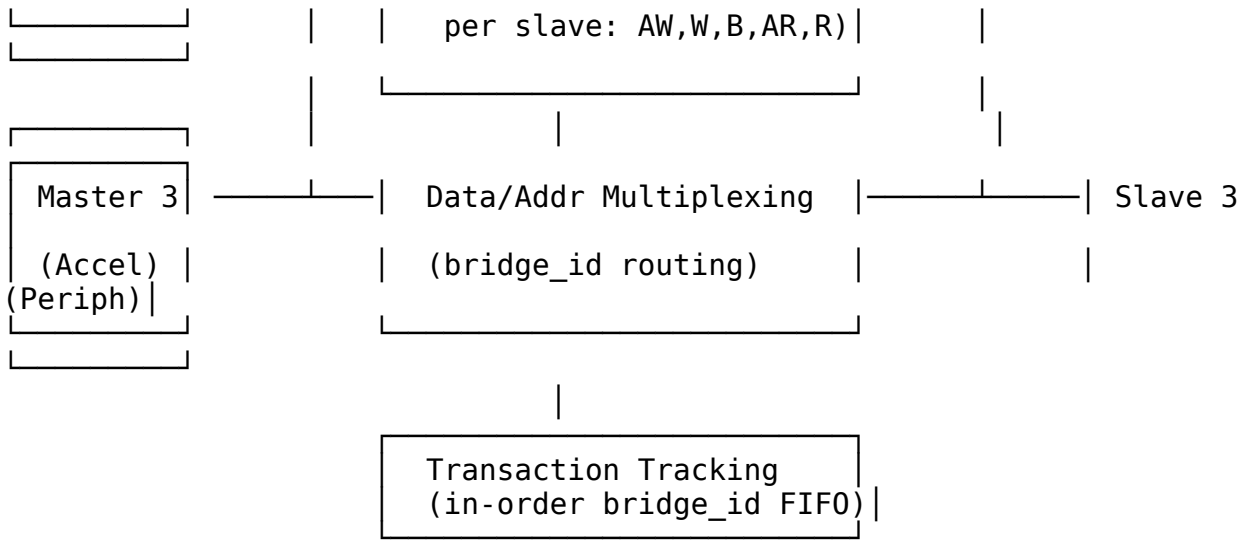
4.5.1 2.1 AXI4 vs AXIS vs APB

Feature	APB	AXI-Stream (Delta)	AXI4 (Bridge)
Protocol Type	Simple register bus	Streaming data	Memory-mapped burst
Channels	1 (address + data)	1 (data stream)	5 (AW, W, B, AR, R)
Request Generation	Address range decode	TDEST decode	Address range decode
Arbitration	Per-slave, per-cycle	Per-slave, packet-locked	Per-slave, per-address-phase
Out-of-Order	No (sequential)	No (streaming order)	No (in-order, `bridge_id` routed)
Burst Support	No	Packet (via TLAST)	Yes (AWLEN/ARLEN)
Complexity	Low	Medium	**High**
Latency	1-2 cycles	2 cycles	2 cycles each way (measured; HAS Table 5.7)
Use Case	Control registers	Data streaming	Memory-mapped I/O

Bridge Complexity Sources: 1. **5 independent channels**, of which TWO are arbitrated. The crossbar instantiates a round-robin arbiter per slave for AW and AR only; W, R and B follow the selection recorded at the address handshake, so they are routed, not arbitrated. 2. **`bridge_id` sideband routing**, in-order (the AXI ID is not used for routing) 3. **Burst handling** with interleaving constraints 4. **Write response tracking** (match AW with B channel) 5. **Address decode** + **ID muxing** for response routing

4.5.2 2.2 Block Diagram





4.5.3 2.3 Key Components

1. Request Generation - Address range decode for each slave - Generates M×S request matrix per channel (AW, AR) - Similar to APB but more complex (2 address channels)

2. Per-Slave Arbitration - **2 arbiters per slave**, not five: - AW channel arbiter (write address) - AR channel arbiter (read address) - W has no arbiter – a W-owner FIFO gives the channel to whichever master's AW the slave accepted first. - B and R have no arbiter – they are muxed combinationally on the `bridge_id` carried alongside the response. - Round-robin, grant held to the ADDRESS handshake – NOT to xlast. The generated arbiters read `lock until handshake`. - Separate read/write paths (no head-of-line blocking)

3. Data Multiplexing - Mux master signals to selected slave - ID-based response routing (B, R channels) - Burst tracking (grant held to the ADDRESS handshake, not to xlast – the generated arbiters read `lock until handshake`; see FR-6)

4. Transaction Tracking - In-order `bridge_id` FIFO per slave adapter (`Bridge ID Tracking - FIFO Mode (In-Order)`), popped in AW/AR order - Track the originating master in a per-slave in-order FIFO, popped on the response. There is no {Master ID, Transaction ID} map – routing is by FIFO POSITION and the AXI ID is not consulted. - Required for routing B/R channels back to correct master - ID tables for OUT-OF-ORDER support are not implemented: `bridge_cam.sv` exists but is instantiated in zero generated bridges

5. ~~Optional Performance Counters~~ – NOT implemented, and no config key enables them - Transaction counts per master/slave - Arbitration conflict counts - Latency histograms

---

4.6 3. Functional Requirements

4.6.1 3.1 AXI4 Protocol Compliance

FR-1: Full AXI4 Protocol Support - Support all 5 AXI4 channels: AW, W, B, AR, R - Comply with AMBA AXI4 specification (ARM IHI 0022) - Support burst lengths: 1-256 beats (AWLEN/ARLEN = 0-255) - Support burst types: INCR, WRAP, FIXED - Support burst sizes: 1-128 bytes (AWSIZE/ARSIZE = 0-7)

FR-2: Response Routing (in-order) - Responses are routed by the `bridge_id` sideband, a static per-master tag, popped from an in-order FIFO. The AXI ID plays no part in routing. - Master IDs pass through unchanged – there is no ID extension. `cpu_adapter` drives `cpu_32b_aw.id = fub_axi_awid`, 4 bits in, 4 bits out. - **Constraint this imposes:** the slave must return responses in the order its requests were accepted. A reordering slave misroutes – the FIFO head names the wrong master, so one master receives another's beats and the second starves. Nothing detects this. - Two masters never alias an ID at a slave: inside the fabric every ID is `{master index, master id}` (bridge TASK-005, was BRIDGE-016), so a slave port carries the widest master's id_width plus `$clog2(NUM_MASTERS)` bits – 8-bit masters behind a 16-master bridge give 12-bit IDs at the slaves. The `bridge_id` sideband still routes responses; the prefix is what makes per-ID tracking sound across masters, and every real AXI slave of a multi-master bridge is tracked by ID in `bridge_cam` (bridge BUG-012, was BRIDGE-015). Single-master bridges add no prefix and keep the in-order FIFO.

FR-3: Atomic Operations (the bridge does NOT implement a monitor – see below) - Pass AWLOCK/ARLOCK through to the slave. **The bridge implements no exclusive monitor**: there is no per-slave exclusive tracking and no EXOKAY generation anywhere in the generated RTL – `awlock` is muxed to the granted slave and nothing more. Exclusives work only if the attached slave implements the monitor itself. 13.1 correctly lists the monitor as future work; this requirement previously read as though the bridge provided it. - ~~Track exclusive monitor per slave~~ – NOT implemented; see the corrected bullet above - ~~Generate BRESP/RRESP errors for failed exclusives~~ – NOT implemented

4.6.2 3.2 Address Decoding

FR-4: Configurable Address Map - Support M masters × S slaves - Each slave has base address and size - Non-overlapping address ranges (verified at generation) - Default slave for unmapped addresses (optional)

FR-5: Address Range Configuration

```
# Example address map
address_map = {
    0: {'base': 0x00000000, 'size': 0x40000000, 'name': 'DDR'},      #
1GB
    1: {'base': 0x40000000, 'size': 0x00100000, 'name': 'SRAM'},     #
1MB
    2: {'base': 0x50000000, 'size': 0x01000000, 'name': 'PCIE'},     #
16MB
```

```
    3: {'base': 0x60000000, 'size': 0x00010000, 'name': 'Peripherals'}
# 64KB
}
```

4.6.3 3.3 Arbitration Strategy

FR-6: Round-Robin Arbitration - Fair bandwidth allocation (no starvation) - Separate arbiters for AW and AR channels - Grant is locked only until the ADDRESS handshake (`awvalid && awready`), not until xlast. The RTL comment reads “lock until handshake”. Back-to-back AWs from different masters can therefore be accepted and their W bursts sequenced by the W-owner FIFO. - Round-robin arbitration, HARD-CODED. There is no TOML key and no parameter to change it; “configurable” was aspirational.

FR-7: Read/Write Independence - Separate read and write paths - No head-of-line blocking between read/write - Concurrent read and write to same slave (if slave supports)

4.6.4 3.4 Burst Handling

FR-8: Burst Optimization - Pipelined burst transfers (overlap address and data) - No artificial burst splitting - Full AXI4 burst protocol support

FR-9: Interleaving Constraints - W channel follows the AW owner via a per-slave W-owner FIFO (not a held grant) - R channel routed by in-order `bridge_id` FIFO position, NOT by transaction ID. IDs are pass-through: the slave port is the same width as the master port. `bridge_cam.sv` exists but is instantiated in zero generated bridges. - ID-based interleaving is NOT supported. Each slave port must return B/R in request order across ALL IDs – see bridge BUG-008 (was BRIDGE-010).

4.7 4. Non-Functional Requirements

4.7.1 4.1 Performance

NFR-1: Latency - **Single-beat read:** ≤ 3 cycles (address decode + arbitration + mux) - **Single-beat write:** ≤ 3 cycles (address decode + arbitration + mux) - **Burst transfer:** No additional latency per beat (pipelined)

NFR-2: Throughput - **Concurrent transfers:** different masters to different slaves, yes; ONE master to several slaves, no – see 1.3 - **Burst efficiency:** Line-rate data transfer after address phase - **No artificial stalls:** Crossbar adds no wait states beyond arbitration

NFR-3: Fmax - **Target:** 300-400 MHz on Xilinx UltraScale+ - **Registered outputs:** All slave outputs registered for timing closure - **Pipelineable:** Optional pipeline stages for >400 MHz

4.7.2 4.2 Resource Usage (Measured)

Post-route, out of context, Vivado 2025.1 (HAS 6.4 flow; full tables in HAS 5.3):

Bridge	Part	LUTs	FFs	BRAM / DSP	Timing
`bridge_2x2_rw` (2 x 2, 32-bit AXI4)	Artix-7 100T -1 @ 10 ns	4,615	3,253	0 / 0	met, +0.12 ns
`bridge_2x2_rw_pipe` (registered crossbar)	Artix-7 100T -1 @ 10 ns	5,646	4,227	0 / 0	met, +1.27 ns
`bridge_2x2_axi5_native` (128-bit, MTE + chunking)	Artix-7 100T -1 @ 10 ns	8,007	6,213	0 / 0	met, +0.04 ns
`bridge_4x4_rw` (4 x 4, 32..256-bit)	Kintex-7 325T -2 @ 6.667 ns	29,719	29,508	0 / 0	met, +0.28 ns

No configuration uses block RAM or DSPs: the per-slave bridge_id FIFOs and the CAM are LUT arrays. Data width sets the cost – the 4 x 4 with width converters is six times the 2 x 2 – and on the Artix-7 at 100 MHz any configuration beyond a plain 2 x 2 needs `xbar_pipeline = true` to close.

4.7.3 4.3 Quality Requirements

NFR-4: Code Generation Quality - Lint-clean SystemVerilog (Verilator) - Synthesizable (Vivado, Yosys, Design Compiler) - No vendor-specific primitives (technology-agnostic) - Clear structure (commented, readable)

NFR-5: Verification - CocoTB testbench framework - Transaction-level verification - ~~Out-of-order test scenarios~~ – not applicable; ordering is structural (see FR-9) - Burst interleaving tests - >95% functional coverage

---

4.8 5. Interface Specifications

4.8.1 5.1 AXI4 Master Interfaces (M × 5 channels)

Write Address Channel (AW):

```
input  logic [ADDR_WIDTH-1:0]   s_axi_awaddr  [NUM_MASTERS];
input  logic [ID_WIDTH-1:0]     s_axi_awid    [NUM_MASTERS];
input  logic [7:0]              s_axi_awlen   [NUM_MASTERS];  // Burst
length-1
input  logic [2:0]              s_axi_awsize  [NUM_MASTERS];  // Burst
size
input  logic [1:0]              s_axi_awburst [NUM_MASTERS];  //
INCR/WRAP/FIXED
input  logic                    s_axi_awlock  [NUM_MASTERS];  //
Exclusive access
input  logic [3:0]              s_axi_awcache [NUM_MASTERS];  // Cache
attributes
input  logic [2:0]              s_axi_awprot  [NUM_MASTERS];  //
Protection type
input  logic [3:0]              s_axi_awqos   [NUM_MASTERS];  // QoS
-- routed, not arbitrated on
input  logic [3:0]              s_axi_awregion[NUM_MASTERS];  //
Region -- passed through
input  logic [UW-1:0]           s_axi_awuser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_awvalid [NUM_MASTERS];
output logic                    s_axi_awready [NUM_MASTERS];
```

Write Data Channel (W):

```
input  logic [DATA_WIDTH-1:0]   s_axi_wdata  [NUM_MASTERS];
input  logic [DATA_WIDTH/8-1:0] s_axi_wstrb  [NUM_MASTERS];  // Byte
strobes
input  logic                    s_axi_wlast  [NUM_MASTERS];  // Last
beat
input  logic [UW-1:0]           s_axi_wuser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_wvalid [NUM_MASTERS];
output logic                    s_axi_wready [NUM_MASTERS];
```

Write Response Channel (B):

```
output logic [ID_WIDTH-1:0]     s_axi_bid    [NUM_MASTERS];
output logic [1:0]              s_axi_bresp  [NUM_MASTERS];  //
OKAY/SLVERR/DECERR (EXOKAY never generated -- no exclusive monitor)
output logic [UW-1:0]           s_axi_buser  [NUM_MASTERS];  // USER
-- returned to the master
```

```
output logic                    s_axi_bvalid [NUM_MASTERS];
input  logic                    s_axi_bready [NUM_MASTERS];
```

Read Address Channel (AR):

```
input  logic [ADDR_WIDTH-1:0]   s_axi_araddr  [NUM_MASTERS];
input  logic [ID_WIDTH-1:0]     s_axi_arid    [NUM_MASTERS];
input  logic [7:0]              s_axi_arlen   [NUM_MASTERS];
input  logic [2:0]              s_axi_arsize  [NUM_MASTERS];
input  logic [1:0]              s_axi_arburst [NUM_MASTERS];
input  logic                    s_axi_arlock  [NUM_MASTERS];
input  logic [3:0]              s_axi_arcache [NUM_MASTERS];
input  logic [2:0]              s_axi_arprot  [NUM_MASTERS];
input  logic [3:0]              s_axi_arqos   [NUM_MASTERS];  // QoS
-- routed, not arbitrated on
input  logic [3:0]              s_axi_arregion[NUM_MASTERS];  //
Region -- passed through
input  logic [UW-1:0]           s_axi_aruser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_arvalid [NUM_MASTERS];
output logic                    s_axi_arready [NUM_MASTERS];
```

Read Data Channel (R):

```
output logic [DATA_WIDTH-1:0]   s_axi_rdata  [NUM_MASTERS];
output logic [ID_WIDTH-1:0]     s_axi_rid    [NUM_MASTERS];
output logic [1:0]              s_axi_rresp  [NUM_MASTERS];
output logic                    s_axi_rlast  [NUM_MASTERS];
output logic [UW-1:0]           s_axi_ruser  [NUM_MASTERS];  // USER
-- returned to the master
output logic                    s_axi_rvalid [NUM_MASTERS];
input  logic                    s_axi_rready [NUM_MASTERS];
```

4.8.2 5.2 AXI4 Slave Interfaces (S × 5 channels)

Mirror of master interfaces, with M → S direction reversed.

4.8.3 5.3 Configuration Is Generation-Time, Not Parameters

The generated bridge exposes no parameters. Everything below is fixed when the generator runs and reaches the module through its package:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    input logic aclk, ...
```

Geometry and widths come from the TOML, per port, and appear inside the generated modules as localparams (`localparam ADDR_WIDTH = 32;`, `localparam ID_WIDTH = 4;` in `cpu_adapter`). To change any of them, regenerate – there is nothing to override at elaboration.

An earlier revision of this section listed a parameter block including `MAX_BURST_LEN`, `PIPELINE_OUTPUTS` and `ENABLE_COUNTERS`. **None of those three exist in any generated module**, and neither do the performance counters that `ENABLE_COUNTERS` implied – there is no counter logic anywhere in the generated set.

4.9 6. Performance Modeling

4.9.1 6.1 Analytical Model

Latency Components (Flat Crossbar):

```
Single-Beat Read Latency:
  1. Address Decode:      0 cycles (combinatorial)
  2. Arbitration:         1 cycle  (AR arbiter decision)
  3. Address Mux:         0 cycles (combinatorial)
  4. Slave Access:        (slave-dependent, not crossbar)
  5. Response Mux:        1 cycle  (R channel ID lookup + mux)
  6. Output Register:     1 cycle  (optional, for timing closure)

  Total (no pipeline):    2 cycles
  Total (pipelined):      3 cycles
```

Burst Transfer Throughput:

```
After address phase completes, data transfer is line-rate:
  - W channel: 1 beat/cycle (routed by the W-owner FIFO)
  - R channel: 1 beat/cycle (ID-routed from slave)

Example: 256-beat burst
  Address phase:  2 cycles (measured, one-time)
  Data phase:     256 cycles (line-rate)
  Total:          258-259 cycles
  Efficiency:     98.8%
```

Concurrent Throughput:

```
Maximum concurrent transfers (4×4 crossbar):
  - Read:  4 concurrent (one per master-slave pair)
  - Write: 4 concurrent (one per master-slave pair)
  - Total: 8 concurrent read+write (separate paths)

Throughput @ 100 MHz, 512-bit data:
  Per path:   100 MHz × 512 bits = 51.2 Gbps
  Total:      8 paths × 51.2 Gbps = 409.6 Gbps theoretical
```

4.9.2 6.2 Resource Scaling

LUT Usage Formula (empirical):

```
LUTs ≈ 500 (base) + 150 × M × S + 20 × FIFO_DEPTH × BRIDGE_ID_WIDTH
```

Example (4×4 crossbar, ID_WIDTH=4, bridge_id FIFO depth 16 per slave):

```
LUTs ≈ 500 + 150 × 16 + 20 × 16 × 4
     ≈ 500 + 2,400 + 1,280
     ≈ 4,180 LUTs
```

4.9.3 6.3 Comparison with Other Crossbars

Crossbar Type	Latency	Throughput	Complexity	Use Case
APB	1-2 cycles	Low (serialized)	Low	Control registers
AXI-Stream (Delta)	2 cycles	High (streaming)	Medium	Data streaming
AXI4 (Bridge)	2 cycles each way	High (burst)	**High**	Memory-mapped I/O
AXI4 + Slices	4-6 cycles	High (burst)	Very High	>400 MHz designs

Bridge Sweet Spot: memory-mapped interconnects that need burst efficiency and mixed protocols. NOT a fit where out-of-order completion matters – the fabric is in-order by construction (FR-9, bridge BUG-008 (was BRIDGE-010)).

4.10 7. Generator Architecture

4.10.17.1 Python Generator Structure

```
class BridgeGenerator:
    """AXI4 crossbar RTL generator"""

    def __init__(self, config):
        self.num_masters = config.num_masters
        self.num_slaves = config.num_slaves
        self.data_width = config.data_width
        self.addr_width = config.addr_width
        self.id_width = config.id_width
        self.address_map = config.address_map

    def generate_address_decode(self) -> str:
        """Generate address range decode logic"""
        # For each master × slave, check if address in range
        # More complex than AXIS (uses address), simpler than full
decode

    def generate_aw_arbiter(self, slave_idx) -> str:
        """Generate write address channel arbiter for one slave"""
        # Round-robin arbiter
        # Grant held to the ADDRESS handshake, not to the B response.
        # (This pseudocode described an abandoned lock-until-response
design.)

    def generate_ar_arbiter(self, slave_idx) -> str:
        """Generate read address channel arbiter for one slave"""
        # Round-robin arbiter
        # Grant held to the ADDRESS handshake, not to RLAST.

    def generate_w_channel_mux(self, slave_idx) -> str:
        """Generate write data channel multiplexer"""
        # W channel follows the AW owner recorded in the per-slave W-
owner
        # FIFO. The arbiter grant itself released at the ADDRESS
handshake.

    def generate_b_channel_demux(self, slave_idx) -> str:
        """Generate write response channel demultiplexer"""
        # Route by FIFO POSITION: pop the per-slave bridge_id FIFO.
        # The returned BID is NOT consulted (see 4.2).

    def generate_r_channel_demux(self, slave_idx) -> str:
```

```
        """Generate read data channel demultiplexer"""
        # Route by FIFO POSITION: pop the per-slave bridge_id FIFO on
RLAST.
        # The returned RID is NOT consulted (see 4.2).

    def generate_bridge_id_fifo(self, slave_idx) -> str:
        """Generate transaction ID tracking table"""
        # Maps {slave, transaction_id} → master_id
        # Indexed on AW/AR grant, looked up on B/R response

    def generate_crossbar(self) -> str:
        """Generate complete crossbar module"""
        # Instantiate all arbiters, muxes, demuxes and the per-slave
        # bridge_id FIFOs (there are no ID tables)
```

4.10.2 7.2 Address Map Configuration

Python Configuration:

```
bridge_config = {
    'num_masters': 4,
    'num_slaves': 4,
    'data_width': 512,
    'addr_width': 64,
    'id_width': 4,
    'address_map': {
        0: {'base': 0x00000000, 'size': 0x40000000, 'name': 'DDR'},
        1: {'base': 0x40000000, 'size': 0x00100000, 'name': 'SRAM'},
        2: {'base': 0x50000000, 'size': 0x01000000, 'name': 'PCIE'},
        3: {'base': 0x60000000, 'size': 0x00010000, 'name':
'Peripherals'}
    },
    # NOTE: 'pipeline_outputs' and 'enable_counters' are NOT real
config
    # keys -- config_validator rejects unknown keys, and 5.3 says the
    # counters do not exist. Removed from this example rather than
left
    # for someone to copy.
}
```

Generated Address Decode:

```
// Address decode for each master
always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++)
        aw_request_matrix[s] = '0;

    for (int m = 0; m < NUM_MASTERS; m++) begin
```

```
        if (s_axi_awvalid[m]) begin
            // Slave 0: DDR (0x00000000 - 0x3FFFFFFF)
            if (s_axi_awaddr[m] >= 64'h00000000 &&
                s_axi_awaddr[m] < 64'h40000000)
                aw_request_matrix[0][m] = 1'b1;

            // Slave 1: SRAM (0x40000000 - 0x400FFFFF)
            if (s_axi_awaddr[m] >= 64'h40000000 &&
                s_axi_awaddr[m] < 64'h40100000)
                aw_request_matrix[1][m] = 1'b1;

            // ... slaves 2-3 ...
        end
    end
end
```

4.11 8. Comparison with APB and Delta

4.11.1 8.1 Code Reuse from APB Generator

Similar Components (~70% reuse): - Address range decode logic (same pattern, different signals) - Round-robin arbiter (same algorithm) - Data multiplexing pattern (same approach) - Backpressure handling (similar to PREADY)

New Components for Bridge: - **2 arbiters per slave** (AW and AR). W is owned via a FIFO, B and R are muxed combinationally – no arbiter on any of the three. - **`bridge_id` sideband routing** (B and R channel demuxing) - **In-order transaction tracking** (a FIFO per direction, not an ID table) - **Burst handling** (grant locked to the ADDRESS handshake, not xlast)

Migration Effort from APB: - ~120 minutes (vs ~75 min for AXIS, due to higher complexity) - Most time: response demuxing and the bridge_id FIFOs

4.11.2 8.2 Code Reuse from Delta Generator

Similar Components (~60% reuse): - Python generation framework - Command-line interface - Performance modeling structure - Arbitration pattern (round-robin with locking)

Key Differences: - **5 channels** vs 1 channel (Delta only has TDATA/TVALID/TREADY/TLAST) - **`bridge_id` sideband routing** vs TDEST-based routing - **Address decode** vs TDEST decode (Bridge more complex) - **Transaction tracking** vs packet atomicity (different mechanisms)

4.11.3 8.3 Complexity Comparison

Metric	APB Crossbar	Delta (AXIS)	Bridge (AXI4)
Channels to arbitrate	1	1	5
Request generation	Address ranges	TDEST decode	Address ranges
Response routing	Grant-based	Grant-based	ID-based
Burst support	No	Packet (TLAST)	Yes (AWLEN/ARLEN)
Out-of-order	No	No	No – in-order `bridge_id` FIFO (`bridge_cam` unused)
Transaction	No	No	Yes

Metric	APB Crossbar	Delta (AXIS)	Bridge (AXI4)
tracking			
Lines of Python	~500	~697	**~900** (est.)
Lines of generated SV	~200 (4×4)	~250 (4×4)	**~400** (4×4) (est.)

= Additional complexity in Bridge

4.12 9. Use Cases

4.12.1 9.1 Multi-Core Processor Interconnect

Scenario: 4 CPU cores + GPU accessing DDR + SRAM + Peripherals

```
Configuration:
  Masters:  5 (4 CPUs, 1 GPU)
  Slaves:   3 (DDR, SRAM, Peripherals)
  Data:     512-bit (cache line width)
  Address:  64-bit (large memory space)
  ID:       4-bit (up to 16 outstanding per master)
```

Benefits: - Concurrent access to all slaves - in-order completion (see FR-9 – out-of-order is NOT implemented) - Burst transfers for cache line fills - Separate read/write paths (no head-of-line blocking)

4.12.2 9.2 DMA + Accelerator System

Scenario: DMA engine + 4 accelerators accessing shared memory

```
Configuration:
  Masters:  5 (1 DMA, 4 accelerators)
  Slaves:   2 (DDR, Control registers)
  Data:     512-bit (high-bandwidth DMA)
  Address:  32-bit (limited address space)
  ID:       2-bit (simple ID space)
```

Benefits: - High-bandwidth memory access for DMA - Fair arbitration prevents accelerator starvation - Control register access doesn't block data transfers

4.12.3 9.3 FPGA System Integration

Scenario: MicroBlaze CPU + custom accelerators + memory controllers

```
Configuration:
  Masters:  8 (1 CPU, 7 accelerators)
  Slaves:   4 (DDR, BRAM, AXI GPIO, AXI DMA)
  Data:     128-bit (AXI4 standard width)
  Address:  32-bit (standard FPGA address space)
  ID:       4-bit (moderate outstanding transactions)
```

Benefits: - Standard AXI4 interfaces (Xilinx IP compatibility) - Scalable to many masters/slaves - Performance counters for profiling. **NOT IMPLEMENTED** – no counter logic exists in any generated module.

4.13 10. Testing Strategy

4.13.1 10.1 FUB (Functional Unit Block) Tests

Address Decode Tests: - Verify all address ranges correctly decoded - Test boundary conditions (base, base+size-1) - Test unmapped addresses (error response)

Arbiter Tests: - Round-robin fairness (all masters get turns) - Grant locking to the ADDRESS handshake (not xlast – see FR-6) - Starvation prevention

bridge_id FIFO Tests: - Correct ID → master mapping - ~~Out-of-order transaction handling~~ – not implemented; see FR-9 - Table full condition

Mux/Demux Tests: - Data integrity through crossbar - Response routing to correct master - Concurrent transfers don't interfere

4.13.2 10.2 Integration Tests

Single-Master, Single-Slave: - Basic read/write transactions - Burst transfers (various lengths) - ~~Out-of-order completions~~ – not applicable (in-order fabric)

Multi-Master, Single-Slave: - Arbitration correctness - Fairness verification - Burst interleaving (if supported)

Multi-Master, Multi-Slave: - Concurrent transfers - No crosstalk between paths - Full M×S matrix coverage

Stress Tests: - All masters active simultaneously - Maximum burst lengths - Full ID space utilization - Back-to-back bursts

4.13.3 10.3 Performance Validation

Latency Measurement: - Single-beat read: measure actual vs analytical - Single-beat write: measure actual vs analytical - Compare with specification (≤3 cycles)

Throughput Measurement: - Burst transfer efficiency (should be ~98%) - Concurrent path throughput (should be line-rate × M×S) - Compare with theoretical maximum

Resource Validation: - Synthesize for Xilinx UltraScale+ - Compare LUT/FF usage with estimates - Verify Fmax ≥ 300 MHz

---

4.14 11. Documentation Plan

4.14.1 11.1 Specifications (Before Code)

- ☒ **PRD.md** - This document (complete requirements)
- ☐ **README.md** - User guide with quick start
- ☐ **BRIDGE_VS_APB_GENERATOR.md** - Migration guide from APB
- ☐ **BRIDGE_VS_DELTA_GENERATOR.md** - Comparison with Delta
- ☐ **AXI4_PROTOCOL_GUIDE.md** - AXI4 primer for students

4.14.2 11.2 Performance Analysis (Before Implementation)

- ☒ **models/bridge_model/bridge_model.py** - Analytical model (V1 Flat implemented)
  - Latency calculations (single-beat, burst)
  - Throughput estimates (concurrent paths)
  - Resource estimates (LUT/FF scaling)
- ☐ **bridge simulator** - Discrete event simulation (optional, not implemented)
  - Cycle-accurate modeling
  - Traffic pattern support
  - Validation against RTL

4.14.3 11.3 Code Generation

- ☐ **bin/bridge_generator.py** - Main RTL generator
  - Command-line interface
  - Address map configuration
  - Generated RTL output

4.14.4 11.4 Verification

- ☐ **dv/tests/** - CocoTB testbenches
  - FUB tests (individual blocks)
  - Integration tests (multi-block)
  - Stress tests (corner cases)

4.15 12. Success Metrics

4.15.1 12.1 Functional Completeness

- ☐ Generates working AXI4 crossbar RTL
- ☐ Passes all CocoTB tests
- ☐ Verilator lint clean
- ☐ Xilinx Vivado synthesis clean

4.15.2 12.2 Performance Targets

- ☐ Latency ≤ 3 cycles (measured in simulation)
- ☐ Throughput = line-rate × M×S paths (measured)
- ☐ Fmax ≥ 300 MHz (post-synthesis)

4.15.3 12.3 Educational Value

- ☐ Complete specifications demonstrating rigor
- ☐ Performance models validate requirements
- ☐ Clear code structure (readable generated RTL)
- ☐ Integration examples provided

4.15.4 12.4 Reusability

- ☐ ~70% code reuse from APB generator (measured by LOC)
- ☐ Similar patterns to Delta generator
- ☐ Can be extended (weighted arbitration, QoS, etc.)

4.16 12.2 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`projects/components/fabric-gen-ip/bridge/docs/`

Quick Commands: Use the provided shell scripts:

```
cd projects/components/fabric-gen-ip/bridge/docs
./generate_has_pdf.sh   # Bridge_HAS_v<rev>.docx/.pdf
./generate_mas_pdf.sh   # Bridge_MAS_v<rev>.docx/.pdf
```

The shell scripts will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the bridge_has/bridge_mas index files 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

See: `bin/md_to_docx.py` for complete implementation details

4.17 13. Future Enhancements

4.17.1 13.1 Short-Term (Post-Initial Release)

- ☐ **Optional pipeline stages** - For Fmax >400 MHz
- ☐ **Weighted arbitration** - QoS support
- ☒ **Default slave** - unmapped address handling. Built 2026-09-07: a subtractive catch-all answers DECERR + 0xDEADBEEF, with a sticky status/IRQ cleared by the `unmapped_clear` INPUT PIN. There is no APB access to it – the pin is the whole interface, and wiring it to a register is the integrator's job. See HAS 4.5 and bridge BUG-007 (was BRIDGE-009).
- ☐ **Exclusive monitor** - Full atomic operation support

4.17.2 13.2 Long-Term

- ☐ **Tree topology** - Hierarchical crossbar (like Delta)
- ☐ **AXI4-Lite variant** - Simplified for control registers
- ☐ **ACE support** - Coherent cache interconnect
- ☐ **GUI configurator** - Visual address map setup

4.18 14. Risk Assessment

Risk	Probability	Impact	Mitigation
~~ID table complexity~~	–	–	Moot: no ID tables were built. The realised risk was the opposite – the DOCS described tables for months while the RTL used an in-order FIFO.
Response-ordering assumption	High	High	The fabric REQUIRES each slave to return B/R in request order across IDs and does not check it. Assert on a returned BID/RID that does not match the FIFO head – see bridge BUG-008 (was BRIDGE-010).
Fmax below target	Low	Medium	Optional pipeline stages for timing closure
Resource usage exceeds	Low	Low	Empirical formulas guide expectations
Burst interleaving bugs	Medium	High	Separate test suite for burst scenarios

4.19 15. Project Timeline (Estimated)

Week 1: Specifications and Models - [x] Day 1-2: PRD.md (complete) - [ ] Day 3-4: Performance modeling (analytical) - [ ] Day 5-7: README.md, migration guides

Week 2-3: Core Implementation - [ ] Day 1-3: Address decode + arbiter generation - [ ] Day 4-5: Data mux/demux generation - [x] Day 6-8: response tracking (delivered as a per-slave in-order bridge_id FIFO, not ID tables) - [ ] Day 9-10: Integration and testing

Week 4: Verification and Examples - [ ] Day 1-5: CocoTB testbenches - [ ] Day 6-7: Integration examples

---

4.20 16. Glossary

- **AXI4:** Advanced eXtensible Interface version 4 (AMBA standard)
- **Burst:** Multi-beat transaction (AWLEN/ARLEN > 0)
- **Crossbar:** Full M×N interconnect matrix
- **ID:** AXI transaction identifier. Passed through this bridge unchanged; it is NOT used for response routing here.
- **Out-of-order:** responses returning in a different order than requested. AXI4 permits it between IDs; **this bridge does not support it** and does not detect a slave that does (bridge BUG-008, was BRIDGE-010).
- **xlast:** WLAST (write) or RLAST (read) - last beat indicator

---

Version: 2.1 **Status:** Specification Complete - Ready for Implementation **Next Steps:** Create performance models, then implement generator

Project Bridge - Connecting masters and slaves across the divide

---

4.21 References

- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — backs the design philosophy (simple-performant, hard limits; class cost discussions and the observation that a full crossbar with buffering is costly), per-slave/channel arbitration as a degenerate separable allocator (ch18 §19.3), round-robin as a starvation-free baseline (ch17 §18.4.2), skid buffers as buffered flow control (ch12), and weighted-arbiter starvation bounds (ch17). (Reference archive)
- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — backs independent read/write paths that avoid head-of-line blocking (ch4), deterministic single-stage crossbar deadlock-freedom (ch3), and the need for explicit ID-keyed tables/CAMs to route out-of-order responses correctly (ch3/4 ordering material). (Reference archive)
- Jantsch & Tenhunen (eds.), *Networks on Chip*, Kluwer 2003 — backs the QoS-plus-aging arbitration policy and its bounded starvation argument (ch13). (Reference archive)
- Maaref, *Architecting and Building High-Speed SoCs*, Packt 2022 — backs the AMBA-suite context for read/write channel independence and the AXI ID ordering rules that Bridge's in-order FIFO simplifies (ch3 §4.2). (Reference archive)
- Roy, *Advanced Digital System Design*, Springer 2023 — backs AXI's independent read/write channels (§18.2.2.1) and the decoupling of W from AW with B correlated through the stored transaction ID (§18.2.3). (Reference archive)
- Eschemann, *The FPGA Programming Handbook*, Packt 2023 — backs the secondary, educational framing of AXI4 channel structure and the cost warning that a full crossbar with buffering can become expensive (ch8). (Reference archive)
- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — the AXI4 protocol authority; backs the ordering rules that permit out-of-order completion across IDs and therefore motivate the CAM-style tracking Bridge documents as future work. (Reference archive)

5 Claude Code Guide: Bridge Subsystem

Version: 2.3 **Last Updated:** 2026-09-13 **Purpose:** AI-specific guidance for working with Bridge subsystem

---

5.1 CRITICAL: Read Architecture Documents First

Before making ANY changes to bridge generator or understanding signal flow:

READ: `projects/components/fabric-gen-ip/bridge/bin/GENERATOR_ARCHITECTURE.md` (generator build mechanics, reconciled against the code 2026-09-29) and `projects/components/fabric-gen-ip/bridge/docs/bridge_mas/` (micro-architecture spec; rendered as `docs/Bridge_MAS_v1.7.pdf`)

These documents contain the **definitive bridge architecture** including: - Correct signal flow (wrappers → decoder → converters → crossbar → slaves) - Component purposes and placement - Common misconceptions that previous agents made - Why there's NO fixed crossbar width

If you skip these documents, you WILL make incorrect assumptions.

5.2 Quick Context

What: Bridge - one AXI4/AXI5 crossbar generator, `bin/bridge_generator.py`, taking TOML or CSV configuration. There is no second generator: the separate `bridge_csv_generator.py` was merged into it and no longer exists. **Status:** Phase 2 Complete - CSV generator with channel-specific masters (wr/rd/rw) **Your Role:** Help users configure CSV files, generate bridges, understand architecture, and create tests

Complete Documentation (Read in This Order): 1. `projects/components/fabric-gen-ip/bridge/bin/GENERATOR_ARCHITECTURE.md` ← **START HERE** (generator build mechanics; the design lives in the MAS below) 2. `projects/components/fabric-gen-ip/bridge/PRD.md` ← Product requirements 3. `vault/Tasks/bridge/INDEX.md` ← Task tracking (open / closed / dropped; the old TASKS.md was folded in 2026-09-10) 4. `projects/components/fabric-gen-ip/bridge/docs/bridge_has/bridge_has_index.md` ← Hardware Architecture Spec (rendered: `docs/Bridge_HAS_v1.7.pdf`) 5. `projects/components/fabric-gen-ip/bridge/docs/bridge_mas/bridge_mas_index.md` ← Micro-Architecture Spec (rendered: `docs/Bridge_MAS_v1.7.pdf`)

(The older BRIDGE_ARCHITECTURE.md, BRIDGE_CURRENT_STATE.md, BRIDGE_ARCHITECTURE_DIAGRAMS.md, CSV_BRIDGE_STATUS.md, and docs/bridge_spec/ documents were superseded by the HAS/MAS above.)

5.3 Target Architecture: Intelligent Width-Aware Routing

Core Principle: Direct connections where possible, converters only where needed, no fixed crossbar width.

5.3.1 Efficient Multi-Width Design

```
Master_A (64b)
  ├─ Direct → Slave_0 (64b)           [0 conversions, minimal latency]
  ├─ Conv(64→128) → Slave_1 (128b)    [1 conversion]
  └─ Conv(64→512) → Slave_2 (512b)    [1 conversion]

Master_B (512b)
  ├─ Conv(512→64) → Slave_0 (64b)     [1 conversion]
  ├─ Conv(512→128) → Slave_1 (128b)   [1 conversion]
  └─ Direct → Slave_2 (512b)          [0 conversions, full bandwidth]
```

Router Logic: Address decoder determines target slave, selects correctly-sized path for that master-slave pair.

5.3.2 Why This Architecture

Naive Fixed-Width Approach (Don't Do This):

```
Master (64b) → Upsize(64→256) → Fixed 256b Crossbar → Downsize(256→64)
→ Slave (64b)
```

- TWO conversions for same-width connections (wasteful!)
- Reduced bandwidth on narrow paths
- Unnecessary logic and area
- Higher latency

Intelligent Routing (Target):

```
Master (64b) → Router → Direct Connection → Slave (64b)
```

- ZERO conversions for matching widths
- Full native bandwidth
- Minimal logic
- Lowest latency

5.3.3 Per-Master Output Paths

Each master has N output paths (one per unique slave width it connects to):

```
// Master_A connects to slaves at 64b, 128b, 512b
// Generate 3 output paths:
```

```
logic [63:0]  master_a_64b_wdata;   // For 64b slaves (direct)
logic [127:0] master_a_128b_wdata;  // For 128b slaves (via converter)
logic [511:0] master_a_512b_wdata;  // For 512b slaves (via converter)

// Router selects based on address decode:
always_comb begin
    case (decoded_slave_id)
        SLAVE_0: select master_a_64b_wdata;   // Slave_0 is 64b
        SLAVE_1: select master_a_128b_wdata;  // Slave_1 is 128b
        SLAVE_2: select master_a_512b_wdata;  // Slave_2 is 512b
    endcase
end
```

5.3.4 Benefits

1. **Resource Efficient** - Only instantiate converters actually needed
2. **Maximum Performance** - Direct paths have zero conversion overhead
3. **Optimal Bandwidth** - No artificial width bottlenecks
4. **Lower Latency** - Minimal logic in critical path for matching widths
5. **Scalable** - Works for any combination of master/slave widths

5.3.5 Implementation Status

Current State: Intelligent per-master multi-width routing – DELIVERED. `cpu_master_adapter.sv` in `bridge_4x4_rw` carries separate 32b, 64b, 128b and 256b paths and converts only where a master and its slave disagree. There is no fixed internal crossbar width.

This block described the Phase-1 fixed-width crossbar as current and the per-width routing as a future migration “requiring generator architecture rework”. That rework has happened; the block outlived it.

5.4 CRITICAL RULE #0: RTL Regeneration Requirements

Change any generator, delete every generated bridge, regenerate all of them. The rule itself, why partial regeneration fails silently, and the symptom list are the root `/CLAUDE.md` Rule #0 and `vault/handbook/design/generated-rtl-discipline.md`. What is bridge-specific:

Generators whose change forces a full regeneration - `bin/bridge_generator.py` - **any** Python file under `bin/bridge_pkg/` (components/, generators/, config, csv_parser, signal_naming, ...) - any Jinja template in `bin/bridge_pkg/jinja_templates/`

The workflow

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean          # removes rtl/generated/
make all            # regenerates from bridge_batch.csv

cd ../dv/tests
make clean-all && make run-all-func    # ALL tests; bridge runs
serial, ~1GB each
```

`clean-all` is not optional here. You have just regenerated every `.sv`, so a stale build directory reports green for the old design – precisely the failure this workflow exists to prevent (`vault/handbook/dv/running-regressions.md`).

Never regenerated: `rtl/bridge_cam.sv` (hand-written, and instantiated by zero generated bridges) and anything in `rtl/` outside `rtl/generated/`.

Identifying a generated file takes five strings, not one. Each generator class stamps its own header, and `bridge_generator.py` appears in none of them:

File	Header
`<name>.sv`	`// Generated by: BridgeModuleGenerator`
`<name>_pkg.sv`	`// Generated by: PackageGenerator`
`<name>_xbar.sv`	`// Generated by: CrossbarGenerator`
`<master>_adapter.sv`	`// Generated by: AdapterGenerator`
`<slave>_adapter.sv`	`// Generated by: SlaveAdapterGenerator`

Grepping for `bridge_generator.py` finds nothing; grepping only for `BridgeModuleGenerator` finds one file in five.

5.5 Critical Pitfalls: slave_select_* Reversion After Handshake

5.5.1 The Problem: Combinational Address Decode

The master adapter's R/B response MUX and the crossbar's AR/AW gating both relied on `<master>_slave_select_{ar,aw}` — a combinational decode of `fub_axi_{ar,aw}addr`. This signal reverts immediately when the address port is freed:

1. AR/AW handshake completes (arvalid && arready, awvalid && awready)
2. Wrapper pops skid buffer and releases the address port
3. fub_axi_araddr / fub_axi_awaddr reverts to next transaction or zero
4. Decode flips to different slave
5. Response MUX is now gated on WRONG slave
6. Response path closes, transaction hangs

This happens BEFORE the converter even finishes pushing to the crossbar, and ALWAYS before response arrives.

5.5.2 Visible Symptoms

- Fails: Multi-slave multi-master bridges hang (timeout)
- Fails: Multi-width masters fail basic connectivity
- Fails: APB/AXIL slaves consistently timeout (longer converter latency = more decode changes)
- Works: Single-slave or single-master systems (no decode change)

5.5.3 The Solution (MANDATORY in All Generators)

For Crossbar AR/AW Gating: - Replace `<master>_slave_select_ar/aw` gates with **inline address re-decode** on stable m_axi buses - Address on m_axi is held stable across the entire AXI handshake

For Master Adapter Response MUX: - Add **per-channel response-tracking FIFOs** that capture slave index at request time - Use FIFO head for response MUX instead of live combinational decode

Crossbar Code Pattern:

```
// WRONG (old implementation):
assign gate_ar_s0 = m0_slave_select_ar && m0_arvalid;
//                  ^^^^^^^^^^^^^^^^^^ Reverts after handshake!

// CORRECT (new implementation):
assign gate_ar_s0 = ((m0_m_axi_araddr >= S0_BASE) &&
                     (m0_m_axi_araddr < S0_END) &&
```

```
                        m0_arvalid);  // Re-decode on stable m_axi
address
```

Master Adapter Code Pattern:

```
// WRONG (old implementation):
assign r_slave_select = comb_slave_select_ar;  // Stale after pop!

// CORRECT (new implementation):
// Capture at request time
always_ff @(posedge aclk) begin
    if (fub_axi_arvalid && fub_axi_arready) begin
        ar_trk_fifo <= comb_slave_select_ar;
    end
end

// Use stable value for response MUX
assign r_slave_select = ar_trk_fifo_out;
```

5.5.4 When This Bug Appears

- Works: Single-master, single-slave systems (no address re-evaluation)
- Fails: Multi-master to same slave (multiple addresses decode differently)
- Fails: Multi-slave same master (address decode changes per transaction)
- Fails: Multi-width paths (W beats re-evaluate AW decode mid-burst)

5.6 MANDATORY: Project Organization Pattern

THIS SUBSYSTEM MUST FOLLOW THE RAPIDS/AMBA ORGANIZATIONAL PATTERN - NO EXCEPTIONS

5.6.1 Required Directory Structure

```
projects/components/fabric-gen-ip/bridge/
├── bin/                            # Bridge-specific tools
(generators, scripts)
│   ├── bridge_generator.py       # AXI4 crossbar generator (CSV/TOML
driven)
│   ├── bridge_pkg/                 # Generator implementation package
│   ├── test_configs/               # TOML port configs + CSV
connectivity
│   └── Makefile                    # make all / make single / make
clean
├── docs/                           # Design documentation
(bridge_has/, bridge_mas/, PDFs)
├── dv/                             # Design Verification (MANDATORY
structure)
│   ├── tbclasses/                  # Testbench classes (MANDATORY - TB
classes here!)
│   │   ├── __init__.py
│   │   └── bridge2x2_rw_tb.py      # Per-config generated TB classes
│   ├── components/                 # Bridge-specific BFMs (if needed)
│   ├── scoreboards/                # Bridge-specific scoreboards (if
needed)
│   ├── testplans/                  # Test plans
│   └── tests/                      # All test files (test runners
only, flat)
│       ├── conftest.py             # Pytest configuration (MANDATORY)
│       └── test_bridge_*.py        # Per-config tests
(test_bridge_2x2_rw.py, ...)
├── rtl/                            # RTL source files
│   ├── bridge_cam.sv               # Hand-written CAM (unused by every
generated bridge)
│   ├── filelists/                  # Generated filelists
│   └── generated/                  # Generated bridges (one subdir per
config)
├── CLAUDE.md                       # This file
├── PRD.md                          # Product requirements
└── TASKS.md                        # Task history and status
```

5.6.2 Testbench Class Location (MANDATORY)

TB classes live in `dv/tbclasses/`, never inside a test runner. The rule and the reasoning are in `vault/handbook/dv/tb-structure.md` and `/GLOBAL_REQUIREMENTS.md` 2.1. The bridge layout that satisfies it is the tree under Rule #0.1 below.

5.6.3 Test File Pattern (MANDATORY)

Pattern B – `cocotb_test_*` functions with pytest wrappers at the bottom, each naming its `testcase=` – is the `test-patterns` skill and `/GLOBAL_REQUIREMENTS.md` 3.2. Read the real thing rather than a template: `dv/tests/test_bridge_2x2_rw.py`. Tests and TB classes are generated per bridge configuration alongside the RTL, so the generated file is the pattern.

5.7 Critical Rules for This Subsystem

5.7.1 Rule #0.1: Testbench Architecture - MANDATORY SEPARATION

TB classes live in `dv/tbclasses/`, one per config (`bridge2x2_rw_tb.py`); runners stay flat in `dv/tests/`. That layout is the tree above – it is not repeated here. Why the separation matters for bridge specifically is that one TB is shared by the per-config test, the `*_mon_monitor.py` stress test and external imports: `vault/handbook/dv/tb-structure.md`.

5.7.2 Rule #1: TBBase and the three mandatory methods

(Former Rule #2 is folded in here; #3 and #4 keep their numbers so existing references to them still resolve.)

Every TB inherits `TBBase` and implements `setup_clocks_and_reset` / `assert_reset` / `deassert_reset`. The requirement is `/GLOBAL_REQUIREMENTS.md` 2.2 and 2.3; what TBBase provides and why the three methods are mandatory is `vault/handbook/dv/tb-structure.md`.

Bridge-specific: the class is `BridgeAXI4FlatTB(TBBase)`, the clock is `aclk` and reset is `aresetn` (active-low), so `assert_reset` drives `self.dut.aresetn.value = 0`.

5.7.3 Rule #3: Use GAXI Components for Protocol Handling

Use the framework BFMs; never hand-drive AXI4 valid/ready. The map and trap list are in `vault/handbook/dv/bfm-usage.md`. Bridge wires GAXI onto the AXI4 channels with the `s0_axi4_` prefix:

```
from CocoTBFramework.components.gaxi.gaxi_master import GAXIMaster
from CocoTBFramework.components.gaxi.gaxi_slave import GAXISlave
from CocoTBFramework.components.axi4.axi4_field_configs import
AXI4FieldConfigHelper

# CORRECT: Use GAXI for AXI4 channels
self.aw_master = GAXIMaster(
    dut=dut,
    title="AW_M0",
    prefix="s0_axi4_",
    clock=clock,

field_config=AXI4FieldConfigHelper.create_aw_field_config(id_width,
addr_width, 1),
    pkt_prefix="aw",
    multi_sig=True,
```

```
    log=log
)
```

5.7.4 Rule #4: Queue-Based Verification

For simple in-order verification, use direct monitor queue access:

```
# CORRECT: Direct queue access
aw_pkt = self.aw_slave._recvQ.popleft()
w_pkt = self.w_slave._recvQ.popleft()

# Verify
assert aw_pkt.addr == expected_addr
assert w_pkt.data == expected_data

# WRONG: Memory model for simple test
memory_model = MemoryModel()  # Unnecessary complexity
```

When to Use Memory Models: - Simple in-order tests → Use queue access - Single-master systems → Use queue access - Complex out-of-order scenarios → not supported; slaves must respond in request order - Multi-master with address overlap → Memory model tracks state

---

5.8 TOML/CSV-Based Bridge Generator (Phase 2 Complete)

5.8.1 Overview

The Bridge generator creates parameterized SystemVerilog crossbars from TOML configuration files with CSV connectivity matrices, eliminating manual RTL editing for complex interconnects.

Key Benefits: - **Human-readable configuration** - TOML for ports, CSV for connectivity - **Custom signal prefixes** - Each port has unique prefix (rapids_m_axi_, apb0_, etc.) - **Channel-specific masters** - Write-only (wr), read-only (rd), or full (rw) - **Interface modules** - Timing isolation via axi4_master/slave wrappers with configurable skid depths - **Automatic converters** - Width and protocol conversion inserted automatically - **Resource efficient** - Only generates needed channels and converters

5.8.2 Quick Start

1. Create bridge_mybridge.toml:

```
[bridge]
  name = "bridge_mybridge"
  description = "Custom bridge example"

  # Default skid buffer depths (can be overridden per port)
  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "cpu", prefix = "cpu_m_axi", id_width = 4, addr_width =
32, data_width = 64, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "dma", prefix = "dma_m_axi", id_width = 4, addr_width =
32, data_width = 512, user_width = 1,
     interface = {type = "axi4_master", skid_depths = {ar = 4, r = 8,
aw = 4, w = 8, b = 4}}}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", id_width = 4, addr_width =
32, data_width = 512, user_width = 1,
     base_addr = 0x00000000, addr_range = 0x80000000, interface =
{type = "axi4_slave"}},
    {name = "sram", prefix = "sram_s_axi", id_width = 4, addr_width =
32, data_width = 256, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x80000000, interface =
{type = "axi4_slave"}}
  ]
```

2. Create bridge_mybridge_connectivity.csv:

```
master\slave,ddr,sram
cpu,1,1
dma,1,1
```

3. Generate bridge:

```
cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_mybridge.toml
# Auto-finds bridge_mybridge_connectivity.csv

# Or use bulk generation
python3 bridge_generator.py --bulk bridge_batch.csv
```

Result: Complete SystemVerilog module with: - Custom port prefixes per port - Timing isolation via interface wrappers - Only needed AXI4 channels (wr/rd/rw optimized) - Width converters for data mismatches - Internal crossbar instantiation - APB/AXI4-Lite converter integration points

5.8.3 Configuration Format Details

TOML Port Configuration:

The primary format is now **TOML** (preferred over CSV for better structure and interface configuration):

Port Specifications: - `name` - Unique identifier (cpu, dma, ddr, etc.) - `prefix` - Signal prefix (cpu_m_axi_, ddr_s_axi_, etc.) - `id_width` - AXI4 ID width in bits - `addr_width` - Address width in bits - `data_width` - Data width in bits (32, 64, 128, 256, 512) - `user_width` - AXI4 user signal width - `base_addr` - Slave base address (slaves only) - `addr_range` - Slave address range (slaves only) - `interface` - Interface wrapper configuration (optional) - `type` - “axi4_master”, “axi4_slave”, “axi4_master_mon”, “axi4_slave_mon”, or omit for direct connection - `skid_depths` - Per-channel buffer depths: {ar, r, aw, w, b}. Valid range is **2..8 inclusive, any integer** – odd depths are legal. `gaxi_skid_buffer` says so at its parameter: “Must be 2..8 inclusive”. An earlier `{2, 4, 6, 8}` here was an inference, not the contract.

CSV Connectivity Matrix:

```
master\slave,slave0,slave1,slave2
master0,1,0,1
master1,0,1,1
```

- `1` = connected, `0` = not connected
- Partial connectivity supported (not all masters to all slaves)
- Auto-detected based on TOML filename: `bridge_name.toml` → `bridge_name_connectivity.csv`

Legacy CSV Format:

For backwards compatibility, the generator still supports CSV port files: - See `bin/test_configs/README.md` for migration guide from CSV to TOML - TOML is now preferred for new bridges (better structure, interface config support)

5.8.4 Channel-Specific Masters (Phase 2 Feature)

Why Channel-Specific? Real hardware often has dedicated read or write masters. Generating all 5 AXI4 channels wastes resources:

Traditional (wasteful):

```
// Write-only master gets unused read channels
input  logic [63:0]  rapids_descr_m_axi_awaddr,  // USED
input  logic [511:0] rapids_descr_m_axi_wdata,   // USED
output logic [7:0]   rapids_descr_m_axi_bid,     // USED
input  logic [63:0]  rapids_descr_m_axi_araddr,  // UNUSED (50%
waste!)
output logic [511:0] rapids_descr_m_axi_rdata,   // UNUSED
```

Channel-Specific (optimized):

```
rapids_descr_wr,master,axi4,wr,rapids_descr_m_axi_,512,64,8,N/A,N/A
```

Generated:

```
// Write-only master - only AW, W, B channels
input  logic [63:0]  rapids_descr_m_axi_awaddr,
input  logic [511:0] rapids_descr_m_axi_wdata,
output logic [7:0]   rapids_descr_m_axi_bid,
// NO READ CHANNELS (araddr, rdata, etc.)
```

Resource Savings: - 40-60% fewer ports for dedicated masters - Only necessary width converters instantiated - Channel-aware direct connection wiring - Faster synthesis, smaller netlists

5.8.5 Example: RAPIDS-Style Configuration

RAPIDS Architecture: - Descriptor write master (wr) - Writes descriptors to memory - Sink write master (wr) - Writes incoming packets to memory - Source read master (rd) - Reads outgoing packets from memory - CPU master (rw) - Full access for configuration

TOML Configuration (bridge_rapids.toml):

```
[bridge]
  name = "bridge_rapids"
  description = "RAPIDS accelerator bridge"
  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "rapids_descr_wr", prefix = "rapids_descr_m_axi", channels
= "wr",
```

```
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "rapids_sink_wr", prefix = "rapids_sink_m_axi", channels =
"wr",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "rapids_src_rd", prefix = "rapids_src_m_axi", channels =
"rd",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "cpu", prefix = "cpu_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 64, user_width = 1,
     interface = {type = "axi4_master"}}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x80000000,
     interface = {type = "axi4_slave"}},
    {name = "apb_periph", prefix = "apb0_", protocol = "apb",
     addr_width = 32, data_width = 32,
     base_addr = 0x00000000, addr_range = 0x00010000}
  ]
```

Connectivity CSV (bridge_rapids_connectivity.csv):

```
master\slave,ddr,apb_periph
rapids_descr_wr,1,0
rapids_sink_wr,1,0
rapids_src_rd,1,0
cpu,1,1
```

Generated RTL Features: - rapids_descr_wr: write channels only - rapids_sink_wr: write channels only - rapids_src_rd: read channels only

The specific counts this list used to give (37 wr / 24 rd against 61 full, for 39% and 61% reductions) reproduce from no port structure in the tree and are not recoverable – the RAPIDS bridges they describe are not in the generated set here. For scale, counted on the shipped variants: an AXI4 `rw` master port is 44 signals, a write-only master 24, a read-only master 20. Channel trimming is real and worth doing; the old percentages were not measurements. - cpu_master: Width converters for 64b→512b upsize (both wr and rd converters) - ddr_controller: Direct 512b connection (no conversion) - apb_periph0: APB slave – `axi4_to_apb4_shim`, a real conversion

5.8.6 Common User Questions

Q: “How do I generate a bridge?”

A: Three steps:

1. Create TOML port configuration file
2. Create CSV connectivity matrix
3. Run bridge_generator.py

```
# Create bridge_mybridge.toml (port configuration)
# Create bridge_mybridge_connectivity.csv (connectivity matrix)

cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_mybridge.toml
# Auto-finds bridge_mybridge_connectivity.csv

# Or use bulk generation for multiple bridges
python3 bridge_generator.py --bulk bridge_batch.csv
```

Q: "What's the difference between wr/rd/rw channels?"

A: Number of AXI4 channels generated:

- **rw** (read/write) - All 5 channels: AW, W, B, AR, R
- **wr** (write-only) - 3 channels: AW, W, B (no read channels)
- **rd** (read-only) - 2 channels: AR, R (no write channels)

Benefits: 40-60% fewer ports for dedicated masters, less logic, faster synthesis

Q: "Can I add timing isolation to ports?"

A: Yes, use interface configuration:

```
masters = [
  {name = "cpu", prefix = "cpu_m_axi", ...,
   interface = {type = "axi4_master", skid_depths = {ar = 2, r = 4, aw
= 2, w = 4, b = 2}}}
]
```

Available interface types: - `"axi4_master"` - Timing isolation on master port - `"axi4_slave"` - Timing isolation on slave port - `"axi4_master_mon"` - Timing + monitoring - `"axi4_slave_mon"` - Timing + monitoring - Omit `interface` field for direct connection

Q: "Can I mix AXI4 and APB slaves?"

A: Yes, use protocol field in TOML:

```
slaves = [
  {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4", ...},
  {name = "apb0", prefix = "apb0_", protocol = "apb", ...}
]
```

Generator inserts `axi4_to_apb4_shim` automatically. This is a real, complete conversion – burst decomposition, width slicing and response mapping – not a Phase-3 placeholder.

Q: “Can ports be AMBA5 (AXI5 / APB5)?”

A: Yes — per port; every wrapper feature rides the fabric natively (bridge TASK-002 (was BRIDGE-002), bridge TASK-004 (was BRIDGE-014), bridge TASK-007 (was BRIDGE-018)):

```
[[bridge.masters]]
protocol = "axi5"
axi5_features = ["trace", "atomic"]  #
nsaid/trace/mpam/mecid/unique/chunking are
                                     # droppable sideband;
poison/atomic/mte are
                                     # connectivity-gated (every
connected path
                                     # must be AXI5 + feature + width-
matched);
                                     # mte and chunking need 128-bit
ports;
                                     # wakeup = AWAKEUP (not path
sideband)

[[bridge.slaves]]
protocol = "apb5"   # APB4 rules apply (rw-only, 32-bit); adds the
APB5 pins
```

Sideband passes natively on width-matched AXI5<->AXI5 paths and terminates with a generation-time warning elsewhere. Read-return atomics forward natively on rw master ports (per-ID return tracker at the slave adapter); on write-only ports the boundary’s `axi5_atomic_filter` answers them with DECERR on B only (its owed R beats are drained inside the filter – a write-only port has no R channel; managers must not issue load-class atomics there). BIDUNQ/RIDUNQ are regenerated per ID at the master boundary, never routed. AXI5-Lite and APB5 are legal as MASTER protocols too (front-end converters). Details: `docs/bridge_has/ch04_interfaces/04_axi5_apb5_interfaces.md` and `docs/bridge_mas/ch02_blocks/10_amba5_boundary.md`.

Q: “Wishbone?” `protocol = "wb4"` on either side (bridge TASK-008 (was BRIDGE-019), HAS 4.6): `channels = "rw"`, `id_width = 0`, 8/16/32/64-bit. One Wishbone transfer is one single-beat AXI4 transaction; AXI bursts decompose to single Wishbone transfers; burst hints are carried, not formed. Best effort by decision – do not propose burst formation or a WB4 CDC port without a consumer.

Q: “What if data widths don’t match?”

A: Generator inserts width converters automatically:

```
masters = [
  {name = "cpu", data_width = 64, ...},  # 64b master
]
slaves = [
  {name = "ddr", data_width = 512, ...}  # 512b slave
```

```
]
# Generator creates width converter: 64b → 512b
```

Q: "Do all masters need to connect to all slaves?"

A: No, use partial connectivity in CSV:

```
master\slave,ddr,sram,apb
rapids_descr,1,1,0     # Connects to ddr and sram only
cpu,1,1,1              # Connects to all three
```

5.8.7 Generator Output Structure

Generated File Contains:

1. **Module Header** - NOT parameterized. The generator resolves every width and count at emission time, so the top module has no NUM_MASTERS / NUM_SLAVES parameters to override.
2. **Port Declarations** - custom prefix per port, channel-specific signals
3. **Internal Signals** - crossbar interface nets
4. **Width Converters** - master-side upsize instances (channel-aware)
5. **Crossbar Instance** - internal AXI4 crossbar
6. **Slave Adapters** - one per slave, carrying the in-order `bridge_id` FIFO
7. **Subtractive slave** - `axi4_subtractive_slave`, ALWAYS emitted, wired to the decode `else`; answers unmapped addresses with DECERR (HAS 4.5)
8. **Protocol shims** - `axi4_to_apb4_shim` / `axi4_to_axil4_*` for apb/axil slaves. These are real conversions, not the Phase-3 TODO placeholders this list used to claim.
9. **Monitor subsystem** (monitored variants) - per-port wrappers, a `monbus_arbiter` tree and one `monbus_axil4_axil4_group`

Example Output Size: - Simple 2x2 bridge: 1,237 lines in the top file; 4,651 across all generated files for that variant (measured, not estimated). Formerly "~400" (plus per-master and per-slave adapters, a crossbar and a package). The "~400" this line used to claim predates monitors, the cfg subsystem and sideband routing. - Complex 5x3 with mixed protocols: 1,776 in the top file; 7,556 across all files. The old "~900" was smaller than the 2x2 figure beside it, which alone showed both were guesses. - Includes comprehensive comments and structure

5.8.8 Testing Generated Bridges

For Pure AXI4 Bridges (Working Now):

```
# Generate bridge (outputs to rtl/generated/<bridge_name>/)
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml
```

```
# Run its test (per-config TB classes live in dv/tbclasses/, e.g.
bridge2x2_rw_tb.py)
cd ../dv/tests
make clean-all && make run-bridge_2x2_rw-gate-parallel   # or -func /
-full, [-waves]
```

Tests are GENERATED too. `make regen` in `bin/` regenerates RTL only; the tests and TB classes come from `bridge_generator.py --bulk bridge_batch.csv --generate-tests`, which writes into `dv/tests` and `dv/tbclasses` per the manifest's own output columns (the `--output-*` flags do not override a bulk run). Regenerate them the same way you regenerate RTL: delete every file marked "Generated by bridge test generator", regenerate all, never hand-edit one. Hand-written tests for a generated DUT live in their own file beside it (`test_bridge_2x2_rw_tracking.py`, `test_bridge_*_sideband*.py`, `*_atomics.py`, `*_bfm5.py`).

For Mixed AXI4/APB: the APB path is complete: `axi4_to_apb4_shim` does real burst decomposition, width slicing and response mapping, and six configurations ship APB slaves with tests in the FULL regression. This line claimed placeholders long after the converter shipped.

5.9 Bridge Architecture Quick Reference

5.9.1 Generated Bridge Crossbar Structure

Illustrative shape (see `rtl/generated/bridge_2x2_rw/bridge_2x2_rw.sv` for a real example):

The generated top takes **no parameters at all** – every width is fixed at generation time and reaches the module through its package:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    input  logic aclk,
    input  logic aresetn,

    // Master-side interfaces (slave ports on bridge)
    // AW, W, B, AR, R channels for each master

    // Slave-side interfaces (master ports on bridge)
    // AW, W, B, AR, R channels for each slave
);
```

5.9.2 Key Features

1. **5-Channel Implementation:** Complete AW, W, B, AR, R routing
2. **Round-Robin Arbitration:** Per-slave arbitration with grant locking
3. **Response routing:** a small in-order FIFO per direction in each slave adapter, holding the `bridge_id` of the master whose request was accepted. Not a RAM, and not indexed by AXI ID.
4. **`bridge_id` sideband routing:** responses return IN ORDER. The slave must complete in the order its requests were accepted; a reordering slave misroutes silently (see FR-2 in the PRD).
5. **Configurable Parameters:** NxM topology, data/addr/ID widths

5.9.3 Address Map

There is no default address map and no 0x10000000 stride. Windows come from each slave's `base_addr` / `addr_range` in the TOML and are baked into the decode. `bridge_2x2_rw` splits the space in half:

- Slave 0 (ddr): 0x00000000 - 0x7FFFFFFF
- Slave 1 (sram): 0x80000000 - 0xFFFFFFFF

Probing 0x1000_0000 expecting "slave 1" hits slave 0.

5.10 Test Organization

Runners are flat in `dv/tests/`: `test_bridge_<config>.py` per configuration, `test_bridge_*_mon_monitor.py` for monitor-stress builds, plus `monitor_stress_common.py` and `conftest.py`. `make list` enumerates them.

5.10.1 Test Levels

Three real levels, both mechanisms, every test (45 of 45 under `bin/review/check_test_levels.py`): each wrapper's `generate_bridge_levels()` branches on `REG_LEVEL` (GATE 72 / FUNC 144 / FULL 216 cells) and exports `TEST_LEVEL` + `SEED` per cell; the TB reads the depth profile from `dv/tbclasses/bridge_levels.py`. Do NOT stamp `TEST_LEVEL` into `os.environ` from a conftest – cocotb_test lets the environment override `extra_env`, and that ran all 216 cells at one depth once (tooling BUG-004, was TOOL-016).

Per-Config Functional Tests (generated): basic connectivity, boundary probe, arbitration (multi-master). Every AXI5 master port carries an `AXI5ComplianceChecker`; every test asserts zero violations.

Monitor Stress Tests (`*_mon_monitor.py`) (generated): bridges built with monitor interfaces, monbus traffic / WRITE_BP / ERR_BP phases scaled by level.

Hand-written: bridge BUG-009 (was BRIDGE-011) tracking + latency on 2x2; the AMBA5 sign-off tests (sideband values, sideband through arbitration, sideband across a width converter, atomics incl. Compare, AXI5 read BFM sign-off).

5.11 Common User Questions and Responses

5.11.1 Q: "How does the Bridge work?"

A: Direct answer:

The Bridge AXI4 crossbar connects multiple AXI4 masters to multiple slaves:

1. **Address Decode (AW/AR):** Routes master requests to appropriate slave based on address
2. **Write Path:** AW → arbitration → slave, W follows locked grant
3. **Read Path:** AR → arbitration → slave, R returns via the slave adapter's in-order `bridge_id` FIFO
4. **Arbitration:** Per-slave round-robin, AW and AR only, with the grant locked until the ADDRESS handshake – not until burst complete. W is owned via a FIFO; B and R have no arbiter.
5. **Response Routing:** each slave adapter keeps an IN-ORDER FIFO of the originating master's `bridge_id`, pushed on the address handshake and popped on the response. Routing is keyed on FIFO POSITION, not on the returned BID/RID – IDs are pass-through and never widened. There are no ID lookup tables; `bridge_cam.sv` is instantiated in zero generated bridges.

Key Features: - Configurable NxM topology - One fabric clock; an AXI4 slave port declared `cdc = true` runs on its own clock behind `axi4_cdc_{wr,rd}` (HAS 4.5a), and an `apb`/`apb5` slave crosses domains inside its own shim (two async FIFOs) - Crossbar options: `xbar_pipeline = true` (registered slave-side channels – needed on the Artix-7 at 100 MHz beyond a plain 2x2, HAS 6.4) and `arbitration = "qos"` with `qos_aging_shift` (HAS 6.8a) - Out-of-context synthesis of any fixture: `projects/components/fabric-gen-ip/bridge/fpga/` (`make synth BRIDGE=<fixture> PART=<part> CLK_NS=<ns>`), measured tables in HAS 5.3 - Subtractive catch-all: an unmapped address gets DECERR + 0xDEADBEEF and a sticky status/IRQ instead of hanging the master (HAS 4.5)

Deliberately NOT supported – these were claimed here for months and none of them is built: - Out-of-order responses. Ordering is structural: one target slave per master at a time (`aw_gate_ok`), and each slave port must return B/R in request order across all IDs (bridge BUG-008, was BRIDGE-010). - Burst-locked arbitration. The grant releases at the address handshake, as item 4 above already said – this bullet contradicted it directly. - Configurable arbitration policy. Round-robin is hard-coded.

See: - `projects/components/fabric-gen-ip/bridge/PRD.md` - Complete specification - `projects/components/fabric-gen-ip/bridge/bin/bridge_generator.py` - Generator implementation

5.11.2 Q: "How do I generate a Bridge?"

A: Use the bridge_generator.py tool:

```
cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml
# Auto-finds test_configs/bridge_2x2_rw_connectivity.csv
# Or regenerate everything: make all  (bulk from bridge_batch.csv)
```

Generated files: - RTL: `projects/components/fabric-gen-ip/bridge/rtl/generated/<bridge_name>/` (top module, crossbar, adapters, package) - Contains complete 5-channel crossbar with ID tracking

5.11.3 Q: "How do I test a Bridge?"

Import the generated TB for that configuration; the runner is generated beside it.

```
cd projects/components/fabric-gen-ip/bridge/dv/tests
make clean-all && make run-bridge_2x2_rw-gate   # serial by design,
~1GB per test
```

`dv/tests/test_bridge_2x2_rw.py` is the full shape.

5.12 PDF Generation Location

Both books build into `docs/` through the wrapper scripts, which apply the house style:

```
cd projects/components/fabric-gen-ip/bridge/docs
./generate_has_pdf.sh    # Bridge_HAS_v<rev> from bridge_has/
./generate_mas_pdf.sh    # Bridge_MAS_v<rev> from bridge_mas/
```

They call `bin/md_to_docx.py`, which follows the index links, demotes headings and builds the ToC. Pipeline detail: `vault/handbook/authoring/doc-pipeline.md`.

Maintained By: RTL Design Sherpa Project

(The version and date for this file are in the header at the top. A second stamp here said 1.0 / 2025-10-18 while the header said 2.1 / 2025-11-03 – two stamps in one file guarantee one of them is wrong.)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Document Information

6.1 Bridge Generator Architecture Specification

Property	Value
Document Title	Bridge Generator Architecture Specification
Version	1.0
Date	2026-10-12
Status	Released
Classification	Open Source - MIT License

6.2 Revision History

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial GAS release — generator code paths verified against `bridge_generator.py`, `bridge_pkg/config.py`, `bridge_pkg/config_validator.py`, `bridge_pkg/config_loader.py`, and emitted artifacts in `rtl/generated/bridge_mix_e/`.

6.3 Document Purpose

This Generator Architecture Specification (GAS) is the manual for the Bridge **code generator**. It is written for the engineer who must:

- re-run the generator after a bugfix,
- add a new protocol or port option,
- debug a generated module,
- or audit why a particular file was emitted.

It covers the full generator surface: CLI flags, TOML keys, Python data classes, validation rules, emission stages, output files, and the extension points that are safe to touch.

For the hardware that the generator produces, see the **Bridge HAS** and **Bridge MAS**. For product requirements, see **PRD.md**.

6.4 Intended Audience

- RTL engineers modifying or extending the generator
- Verification engineers regenerating fixtures after a generator change
- System architects adding new bridge configurations to the batch
- Integrators debugging a generated bridge top

6.5 Related Documents

- **Bridge HAS** - Hardware Architecture Specification
- **Bridge MAS** - Micro-Architecture Specification
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide and subsystem rules

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

7 What the Generator Produces

The Bridge generator (`bin/bridge_generator.py`) emits a complete, synthesizable AXI4/AXI5 crossbar from a TOML port description and a CSV connectivity matrix. The output is **not** a parameterized Verilog template: every width, count, and protocol conversion is resolved at generation time, and the top module exposes no parameters.

7.1 Generated Artifact Family

For each requested variant (`no` or `mon`), the generator writes a directory under `rtl/generated/<bridge_name>/` containing:

File	Stamped generator
`<bridge_name>_pkg.sv`	`PackageGenerator`
`<bridge_name>.sv`	`BridgeModuleGenerator`
`<bridge_name>_xbar.sv`	`CrossbarGenerator`
`<master>_adapter.sv`	`AdapterGenerator` (per master)
`<slave>_adapter.sv`	`SlaveAdapterGenerator` (per slave)
`<bridge_name>.f`	`bridge_generator.py` (filelist)
`<bridge_name>.toml`	copied source config
`<bridge_name>_connectivity.csv`	copied connectivity matrix
`__cfg_*_pkg.sv` / `<bridge_name>_cfg.sv`	`CfgRdlGenerator` (only when `use_cfg_regblock = true`)

The header comment in each `.sv` file names the generator class that produced it. This is the fastest way to tell whether a file is generated and which part of the pipeline owns it.

7.2 What Each Variant Means

The TOML `[bridge].variants` list selects which variants to emit:

- `"no"` — bare bridge, no monitor wrapper instances.
- `"mon"` — bridge with `_monlite` wrappers on every enabled port, plus the monbus arbiter and an internal AXIL4/AXIL4 group (unless `internal_axil_group = false`).

Both variants are fully independent; each gets its own directory and its own filelist.

7.3 Scope of Generation

The generator resolves:

- Port prefixes, channel subsets (`rw`/`wr`/`rd`), widths, and protocol variants.
- Width converters (`axi4_dwidth_converter_{wr,rd}`) inserted only where master and slave data widths differ.
- Protocol shims (`axi4_to_apb4_shim`, `axi4_to_apb5_shim`, `axi4_to_axil4_*`, `axi4_to_axil5_*`, `axi4_to_axi3`, `axi4_to_wb4`, AXI5 boundary wrappers) selected by the port `protocol` field.
- The subtractive catch-all slave (`axi4_subtractive_slave`), always present and wired to unmapped addresses.
- Optional monitor wrappers and a cfg register block.

The generator does **not** emit testbench code unless `--generate-tests` is passed.

---

8 When to Regenerate

Any change to the generator or its inputs can change every emitted file. Bridge enforces a **full regeneration discipline**.

8.1 Triggers for Full Regeneration

Regenerate every generated bridge in `rtl/generated/` when any of the following change:

- `bin/bridge_generator.py`
- Any Python file under `bin/bridge_pkg/`
- Any Jinja template under `bin/bridge_pkg/jinja_templates/`
- Any shared converter or wrapper IP the filelist pulls in, if its interface changes

Do **not** regenerate a single bridge and leave the rest untouched. Generated files share package definitions, converter selection logic, and filelist conventions; partial regeneration creates version skew.

8.2 Regeneration Workflow

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean          # removes rtl/generated/
make all            # regenerates every bridge in bridge_batch.csv
```

After regenerating RTL, also regenerate tests if the generator's test templates changed:

```
python bridge_generator.py --bulk bridge_batch.csv --generate-tests
```

Then run the DV suite from a clean build directory:

```
cd ../dv/tests
make clean-all && make run-all-func
```

8.3 When Inputs (Not Code) Change

If only the TOML or CSV for one bridge changes, regenerate that bridge alone is safe only when **no generator code changed**:

```
python bridge_generator.py --ports test_configs/my_bridge.toml
```

If in doubt, run `make clean && make all`.

8.4 Identifying Generated Files

Each generator class stamps its own header. Grepping for the generator name is more reliable than grepping for `bridge_generator.py`:

Generator class	Header string
`BridgeModuleGenerator`	`Generated by: BridgeModuleGenerator`
`PackageGenerator`	`Generated by: PackageGenerator`
`CrossbarGenerator`	`Generated by: CrossbarGenerator`
`AdapterGenerator`	`Generated by: AdapterGenerator`
`SlaveAdapterGenerator`	`Generated by: SlaveAdapterGenerator`

Hand-written files in `rtl/` (for example `bridge_cam.sv`) are never regenerated and must not be deleted by `make clean`.

---

9 Generator Entry Points

Bridge has two invocation modes: single-bridge and bulk.

9.1 Single-Bridge Mode

```
cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py \
    --ports test_configs/bridge_2x2_rw.toml \
    [--connectivity test_configs/bridge_2x2_rw_connectivity.csv] \
    [--name bridge_2x2_rw] \
    [--output-dir ../rtl/generated]
```

For TOML inputs, `--connectivity` is optional: the loader auto-detects `<bridge_name>_connectivity.csv` next to the TOML file. CSV port files are no longer supported.

CLI flags:

Flag	Default	Purpose
`--ports`	required	Path to the TOML port configuration
`--connectivity`	auto	Path to the CSV connectivity matrix
`--name`	from TOML or auto	Output module base name
`--output-dir`	`../rtl/generated`	Where to write the bridge directory
`--generate-tests`	off	Also emit cocotb TB class and test runner
`--output-tb`	`../dv/tbclasses`	TB class output directory
`--output-test`	`../dv/tests`	Test runner output directory

9.2 Bulk Mode

```
python3 bridge_generator.py --bulk bridge_batch.csv
```

The batch CSV columns are:

```
name,ports,connectivity,output_dir,output_tb,output_test
```

Comment lines starting with # are ignored. The `name` and `connectivity` fields may be empty; empty connectivity triggers auto-detection from the TOML filename.

9.3 Makefile Targets

The `bin/Makefile` wraps the bulk path:

Target	Action
`make all`	`bridge_generator.py --bulk bridge_batch.csv`
`make clean`	remove `rtl/generated/` and `rtl/filelists/`
`make regen`	alias for `make clean && make all`

9.4 Return Value

`generate_bridge()` returns a tuple `(success: bool, emitted_variants: list[tuple[str, bool]])`. A non-empty `emitted_variants` list means at least one variant was written. The bulk driver exits non-zero if any bridge or test generation fails.

10 Bridge-Level TOML Keys

The TOML `[bridge]` table configures the bridge as a whole. It is parsed by `bridge_pkg.config_loader.load_config()` into the `BridgeConfig` dataclass (`bridge_pkg/config.py`).

10.1 `[bridge]` scalar keys

Key	Type	Default	Dataclass field	Description
`name`	string	`""`	`BridgeConfig.name`	Output base name. If empty, the generator auto-names from topology (`bridge_<M>x<S>_<channels>`).
`variants`	list of strings	`["no"]`	`BridgeConfig.variants`	Build variants to emit. Each entry must be `"no"` or `"mon"`. At least one entry is required in TOML.
`description`	string	`""`	ignored	Human-readable comment; copied into generated header only.

10.2 `[bridge.clock]` table

Key	Type	Default	Used by	Description
`name`	string	`"aclk"`	header comment	Clock signal name (informational).
`frequency_mhz`	int	none	header comment	Clock frequency (informational).

10.3 Crossbar and arbitration keys

Key	Type	Default	Dataclass field	Description
`xbar_pipeline`	boolean	`false`	`BridgeConfig.xbar_pipeline`	Add skid register stage on every slave-side channel inside the crossbar.
`arbitration`	string	`"rr"`	`BridgeConfig.arbitration`	Per-slave arbitration policy. `"rr"` = round-robin; `"qos"` = QoS + aging.
`qos_aging_shift`	int	`4`	`BridgeConfig.qos_aging_shift`	For `"qos"`, a waiting request gains one priority level every `2**shift` cycles.

10.4 Monitor and cfg subsystem keys

Key	Type	Default	Dataclass field	Description
`mon_preset`	string	`"lite"`	`BridgeConfig.mon_preset`	Baseline monitor reporter enables. Accepted aliases: `"all"`, `"error_only"`, `"functional"`, `"none"`, `"lite"`. Since 2026-09-26 every monitored port uses `axi_monitor_lite`, so the preset resolves to the lite set regardless.
`use_all_monitors`	boolean	`false`	`BridgeConfig.use_all_monitors`	Force `USE_MONITOR=1` on every port. Mutually exclusive with `use_no_monitors`.
`use_no_monitors`	boolean	`false`	`BridgeConfig.use_no_monitors`	Force `USE_MONITOR=0` on every port. Mutually exclusive with `use_all_monitors`.
`internal_axil_group`	boolean	`true`	`BridgeConfig.internal_axil_group`	For `"mon"` variants, instantiate the internal `monbus_axil4_axil4_group`. When `false`, expose the aggregated monbus stream at

Key	Type	Default	Dataclass field	Description
				the top.
`use_cfg_regblock`	boolean	`false`	`BridgeConfig.use_cfg_regblock`	Replace the 350+ individual `cfg_*` top-level pins with a PeakRDL-generated cfg register block.

10.5 [bridge.mon_group] table

Key	Type	Default	Dataclass field	Description
slave_protocol	string	"axil4"	MonGroupConfig.slave_protocol	Protocol of the monbus group slave-read port. "axil4" or "axi4". Deprecated spelling "axil" is normalized to "axil4".
master_protocol	string	"axil4"	MonGroupConfig.master_protocol	Protocol of the monbus group master-write port. "axil4" or "axi4".
axi_id_width	int	8	MonGroupConfig.axi_id_width	AXI ID width when either side is axi4.
max_burst_beats	int	64	MonGroupConfig.max_burst_beats	Max write burst when master side is axi4.
fifo_depth_err	int	64	MonGroupConfig.fifo_depth_err	Error FIFO depth.
fifo_depth_write	int	96	MonGroupConfig.fifo_depth_write	Write-data FIFO depth in beats.
flush_timeout_cycles	int	1024	MonGroupConfig.flush_timeout_cycles	Flush timeout.
use_compression	int	0	MonGroupConfig.use_compression	Compression enable.

10.6 `[bridge.defaults]` table

The `defaults.skid_depths` sub-table provides default skid depths for interface wrappers:

```
[bridge.defaults]
skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}
```

Per-port `interface.skid_depths` overrides these defaults. Valid per-channel depths are integers in the range `2..8` inclusive.

---

11 Port Specification Schema

Each master and slave is described by a TOML table under `[[bridge.masters]]` or `[[bridge.slaves]]`. The loader converts each table into a `PortSpec` dataclass (`bridge_pkg/config.py`).

11.1 Common port fields

Field	Type	Default	PortSpec attribute	Description
`name`	string	required	`port_name`	Unique identifier used in generated module names and file names.
`prefix`	string	required	`prefix`	Signal prefix. The generator normalizes a missing trailing underscore (`cpu_m_axi` → `cpu_m_axi_`).
`protocol`	string	required	`protocol`	One of `axi3`, `axi4`, `axi5`, `apb`, `apb5`, `axil`, `axil5`, `wb4`.
`channels`	string	required	`channels`	One of `rw`, `wr`, `rd`. Slaves must declare what they support; the generator trims unneeded channels.
`data_width`	int	required	`data_width`	Data bus width in bits.
`addr_width`	int	required	`addr_width`	Address bus width in bits.
`id_width`	int	`0`	`id_width`	AXI ID width. Required > 0 for `axi4`/`axi5` masters; must be 0 for APB, AXI-Lite, and Wishbone.

Field	Type	Default	PortSpec attribute	Description
`user_width`	int	`0`	(passed to wrapper)	AXI4 user signal width.

11.2 Slave-only fields

Field	Type	Default	PortSpec attribute	Description
base_addr	hex string/ int	required for slaves	base_addr	Start of the slave address window. Must be 4 KiB-aligned.
addr_range	hex string/ int	required for slaves	addr_range	Size of the slave window. Must be a non-zero multiple of 4 KiB.

11.3 Optional behavior fields

Field	Type	Default	PortSpec attribute	Description
interface	table	none	see below	Timing-isolation wrapper configuration.
cdc	boolean	false	cdc	Run this slave port on its own clock behind axi4_cdc_{wr,rd}. Legal only for protocol="axi4" slave ports.
enable_ooo	boolean	false	enable_ooo	Mark slave as out-of-order capable. (No generated fixture currently uses this.)
use_monitor	boolean	true	use_monitor	Per-port monitor enable; meaningful only for "mon" variants.
mon_add	list of strings	[]	mon_add	Additive reporter cones beyond the preset. Names come from MON_CONES: error, timeout, compl, threshold, perf, debug.
mon_remove	list of string	[]	mon_remove	Reporter cones to remove from

Field	Type	Default	PortSpec attribute	Description
	s			the preset.
axi5_features	list of strings	[]	axi5_features	AXI5 sideband feature list. Legal only on axi5 or axil5 ports.
internal	boolean	false	internal	Port has no top-level pins. Used automatically for the subtractive catch-all.

11.4 axi5_features values

The validator (bridge_pkg/config_validator.py) accepts:

- Droppable sideband: nsaid, trace, mpam, mecid, unique, chunking.
- Connectivity-gated (every connected path must be native AXI5, same feature, same width): poison, atomic, mte.

On axil5 ports, only user and exclusive are forwardable; trace, loop, mpam, mecid, nsaid, poison are rejected because they have no AXI4 source/destination.

11.5 Channel semantics

channels	Master gets	Slave must support
rw	AW, W, B, AR, R	AW, W, B, AR, R
wr	AW, W, B only	AW, W, B
rd	AR, R only	AR, R

A wr master may connect to a wr or rw slave; a rd master may connect to a rd or rw slave. A wr master cannot connect to a rd slave, and vice versa.

11.6 Example

```
[[bridge.masters]]
name       = "cpu"
prefix     = "cpu_m_axi_"
protocol   = "axi4"
channels   = "rw"
id_width   = 4
addr_width = 32
data_width = 64
user_width = 1

[[bridge.slaves]]
name       = "ddr"
prefix     = "ddr_s_axi_"
protocol   = "axi4"
channels   = "rw"
id_width   = 6
addr_width = 32
data_width = 512
user_width = 1
base_addr  = "0x00000000"
addr_range = "0x80000000"
```

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

12 Interface Wrapper Configuration

The optional per-port `interface` table selects a timing-isolation wrapper and its skid depths. If omitted, the port is connected directly to the crossbar (no wrapper).

12.1 `interface.type` values

Value	Wrapper instantiated
`axi4_master`	`axi4_slave_wr` / `axi4_slave_rd` (master port acts as AXI slave to external master)
`axi4_slave`	`axi4_master_wr` / `axi4_master_rd` (slave port acts as AXI master to external slave)
`axi4_master_mon`	`_monlite` variant of the master wrapper
`axi4_slave_mon`	`_monlite` variant of the slave wrapper

The `_monlite` wrappers are used unconditionally since 2026-09-26; the old non-lite wrappers were retired.

12.2 `interface.skid_depths` table

Key	Type	Default	Valid range	Description
`ar`	int	from `[bridge.defaults.skid_depths]`	2..8	Read address skid depth.
`r`	int	from defaults	2..8	Read data skid depth.
`aw`	int	from defaults	2..8	Write address skid depth.
`w`	int	from defaults	2..8	Write data skid depth.
`b`	int	from defaults	2..8	Write response skid depth.

Example:

```
[[bridge.masters]]
name = "dma"
prefix = "dma_m_axi_"
protocol = "axi4"
channels = "rw"
id_width = 4
addr_width = 32
data_width = 512
user_width = 1
interface = {type = "axi4_master", skid_depths = {ar = 4, r = 8, aw =
4, w = 8, b = 4}}
```

12.3 How the wrapper selection flows

1. `bridge_pkg/config_loader.py` reads the `interface` table into `PortSpec` attributes.
2. `bridge_generator.py` passes the port protocol and `use_monitor` flag to `BridgeModuleGenerator`.
3. `AdapterGenerator` (`bridge_pkg/generators/adapter_generator.py`) and `SlaveAdapterGenerator` (`bridge_pkg/generators/slave_adapter_generator.py`) instantiate the appropriate wrapper module based on protocol and monitor state.
4. The generated adapter module wires the wrapper's `fub_axi_*` internal face to the crossbar.

12.4 Protocol-specific wrapper notes

- `axi5` ports use `axi5_slave_{wr,rd}` on master sides and `axi5_master_{wr,rd}` on slave sides.
- `apb`/`apb5` master ports use `apb{4,5}_to_axi4` front-end converters inside the adapter.
- `wb4` master ports use `wb4_to_axi4`.
- `axi3` master ports use `axi3_to_axi4`.
- `apb`, `apb5`, `axil`, `axil5`, `wb4`, and `axi3` slave ports use the corresponding completer shim (`axi4_to_apb4_shim`, `axi4_to_wb4`, etc.).

---

13 Monitor and CFG Subsystem Keys

Bridge can optionally emit a monitor infrastructure and a cfg register block. This section documents the keys that control those subsystems.

13.1 Monitor subsystem

The "mon" variant wraps each enabled port with a _monlite timing wrapper. The wrapper exposes a monbus_* packet interface that feeds a per-bridge monbus_arbiter and, by default, a monbus_axil4_axil4_group.

13.1.1 Bridge-level monitor keys

Key	Type	Default	Description
mon_preset	string	"lite"	Baseline reporter enables. Since 2026-09-26 all monitored ports use axi_monitor_lite, so every preset resolves to the same hardware; the field is kept for backward-compatible config loading.
use_all_monitors	boolean	false	Override every port's use_monitor to true.
use_no_monitors	boolean	false	Override every port's use_monitor to false. Mutually exclusive with use_all_monitors.

13.1.2 Per-port monitor keys

Key	Type	Default	Description
use_monitor	boolean	true	Include the _monlite wrapper on this port in a "mon" variant.
mon_add	list	[]	Extra reporter cones to enable beyond the preset.
mon_remove	list	[]	Reporter cones to disable from the preset.

MON_CONES in bridge_pkg/config.py defines the valid cone names: error, timeout, compl, threshold, perf, debug.

13.1.3 `[bridge.mon_group]` table

This table selects the internal monbus aggregation module. See “Bridge-Level TOML Keys” for the full schema. The selected module is:

```
f"monbus_{slave_protocol}_{master_protocol}_group"
```

Examples: `monbus_axil4_axil4_group`, `monbus_axi4_axi4_group`.

When `internal_axil_group = false`, the bridge still instantiates the `monbus_arbiter` but exposes its aggregated stream as `monbus_agg_*` top-level outputs, letting an external consumer provide the AXIL group.

13.2 CFG register block

Setting `use_cfg_regblock = true` replaces the 350+ individual `cfg_<port>_<idx>_<chan>_*` top-level pins with a PeakRDL-generated register block.

13.2.1 What changes

- `CfgRdlGenerator` (`bridge_pkg/generators/cfg_rdl_generator.py`) builds an RDL schema from the adapter list and `BridgeModuleGenerator._MON_GROUP_CFG`.
- PeakRDL emits `<bridge_name>_cfg_pkg.sv` and `<bridge_name>_cfg.sv` into the bridge directory.
- `bridge_generator.py` patches the top module to narrow the 32-bit `s_cfg_axil_*addr` into the regblock's actual address width.
- The filelist gains the cfg package and body, plus a Verilator waiver for PeakRDL's `field_combo MULTIDRIVEN` warning.

13.2.2 Per-port cfg fields

`CfgAdapterPort` is constructed in `bridge_generator.py` for every master and every non-internal slave. Fields include:

Field	Meaning
`name`	Port name
`idx`	Index within direction
`has_wr` / `has_rd`	Which directions exist after channel trimming
`perfmon_enabled`	Whether perf counters exist for this port
`n_addr_ranges`	Number of address ranges (currently 0)
`addr_width`	Address width for this adapter

The subtractive catch-all slave is deliberately excluded from the cfg map so that adding or removing it does not shift register offsets.

14 Validation Rules

Configuration validation is centralized in `bridge_pkg/config_validator.py`. The loader calls `validate_config(masters, slaves, connectivity)` before any RTL is emitted. Validation failures raise `ValidationError` and stop generation.

14.1 Field validation (`validate_required_fields`)

- Every port must have `name` and `prefix`.
- `data_width` and `addr_width` must be positive.
- `axi4` masters must have `id_width > 0`.
- `axil`/`axil5` masters must have `id_width == 0`.
- `apb`/`apb5`/`wb4`/`axi3` masters must have `addr_width == 32`.
- Slaves must have `base_addr` and a positive `addr_range`.
- Slave `base_addr` and `addr_range` must be 4 KiB-aligned / a multiple of 4 KiB.

14.2 Channel and protocol validation

- `channels` must be one of `rw`, `wr`, `rd` (`validate_channels`).
- `protocol` must be one of `axi3`, `axi4`, `axi5`, `apb`, `apb5`, `axil`, `axil5`, `wb4` (`validate_protocol`).
- Slaves must explicitly specify `channels` (`validate_slave_channels_explicit`).

14.3 Protocol-specific constraints

14.3.1 APB / APB5 (`validate_apb_constraints`)

- `channels` must be `rw`.
- `data_width` must be 8, 16, or 32.
- `id_width` must be 0.

14.3.2 Wishbone B4 (`validate_wb4_constraints`)

- `channels` must be `rw`.
- `data_width` must be 8, 16, 32, or 64.
- `id_width` must be 0.

14.3.3 AXI3 (`validate_axi3`)

- AXI3 masters must have `id_width > 0`.
- `data_width` must be 32, 64, 128, or 256.
- CDC is not supported.

14.3.4 AXI5 (`validate_axi5`)

- `axi5_features` may appear only on `axi5` or `axil5` ports.
- Droppable features: `nsaid`, `trace`, `mpam`, `mecid`, `unique`, `chunking`.
- Connectivity-gated features: `poison`, `atomic`, `mte`.
- `mte` and `chunking` require `data_width >= 128` (`validate_axi5_wide_features`).
- `poison`, `atomic`, `mte` require every connected path to be native AXI5 with the feature enabled and the same data width (`validate_axi5_poison_connectivity`).
- An `rw` AXI5 master with `atomic` may only connect to `rw` AXI5 slaves (`validate_axi5_atomic_read_return`).
- `axil5` ports accept only `user` and `exclusive` as forwardable features (`validate_axil5_features`).

14.4 Master-slave compatibility (`validate_master_slave_compatibility`)

- A `wr` master cannot connect to a `rd` slave.
- A `rd` master cannot connect to a `wr` slave.
- Compatibility is checked only for connected pairs (per the CSV connectivity matrix).

14.5 Address map validation (`validate_address_map`)

- Positively-decoded slave windows must not overlap.
- The subtractive catch-all (`internal=true`) is excluded from overlap checks.

14.6 Slave ID width (`validate_slave_id_widths`)

With more than one master, every AXI slave port's ID is widened by `$clog2(NUM_MASTERS)` master-index bits. The validator checks that each slave's declared `id_width` is at least:

```
widest_master_id_width + clog2(num_masters)
```

14.7 CDC constraints (`validate_cdc_constraints`)

- `cdc=true` is legal only on external AXI4 slave ports.
- Internal slaves cannot be CDC ports.

14.8 Warning for dropped sideband (`warn_axi5_dropped_sideband`)

Droppable AXI5 sideband that terminates mid-path prints a generation-time warning naming the path and the reason (non-AXI5 peer, missing feature, or width converter).

15 Pipeline Overview

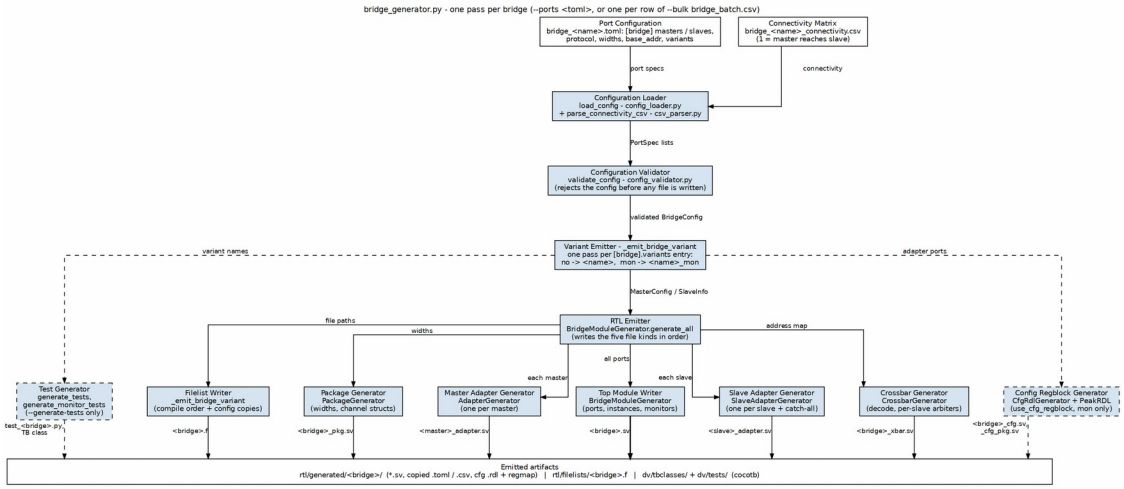
The Bridge generation pipeline has five stages:

1. **Parse** — TOML and CSV files are read into Python data structures.
2. **Validate** — `validate_config()` checks semantic and protocol rules.
3. **Translate** — `PortSpec` objects are converted to `MasterConfig`/`SlaveInfo` objects consumed by the emitter.
4. **Emit RTL** — `BridgeModuleGenerator.generate_all()` writes package, adapters, crossbar, and top module.
5. **Emit collateral** — filelist, copied configs, optional regblock, optional tests.

Each variant (`no`, `mon`) runs stages 3–5 independently and writes to its own directory.

15.1 Pipeline diagram

15.1.1 Figure 3.1: Bridge Generator Pipeline



Bridge generator pipeline

Configuration enters at the top and passes down the center column: load, validate, then one pass per variant into the RTL emitter, which fans out to one sub-generator per file kind. Every output lands on the emitted-artifacts bar at the bottom. The dashed boxes run only when asked for: the cocotb test generator under `--generate-tests`, and the PeakRDL config regblock for a `use_cfg_regblock` bridge's `mon` variant. Nothing in the generator arbitrates, so the figure has no trapezium.

15.2 Key files by stage

Stage	File	Function
Parse	`bridge_pkg/config_loader.py`	`load_config()` reads TOML and CSV.
Validate	`bridge_pkg/config_validator.py`	`validate_config()` runs all checks.
Translate	`bridge_generator.py`	`_emit_bridge_variant()` builds `MasterConfig`/`SlaveInfo`.
Emit RTL	`bridge_pkg/components/bridge_module_generator.py`	`BridgeModuleGenerator.generate_all()` orchestrates sub-generators.
Emit adapters	`bridge_pkg/generators/adapter_generator.py`	`AdapterGenerator` emits master adapters.
Emit slave adapters	`bridge_pkg/generators/slave_adapter_generator.py`	`SlaveAdapterGenerator` emits slave adapters.
Emit crossbar	`bridge_pkg/generators/crossbar_generator.py`	`CrossbarGenerator` emits the internal crossbar.
Emit package	`bridge_pkg/generators/package_generator.py`	`PackageGenerator` emits the SV package.
Emit filelist	`bridge_generator.py`	`_emit_bridge_variant()` writes the `.f` filelist.
Emit tests	`bridge_generator.py`	`generate_tests()` / `generate_monitor_tests()` write cocotb files.
Emit cfg	`bridge_pkg/generators/cfg_rdl_generator.py`	`CfgRdlGenerator` emits the PeakRDL register block.

16 Configuration Loading

Configuration loading is implemented in `bridge_pkg/config_loader.py`. The entry point is `load_config(ports_file, connectivity_file=None)`.

16.1 TOML parsing

The loader uses Python's `tomllib` (or `tomli` on older Pythons) to read the TOML file. It expects:

- A `[bridge]` table with scalar keys and `[[bridge.masters]]` / `[[bridge.slaves]]` arrays.
- An optional `[bridge.mon_group]` table.
- An optional `[bridge.defaults]` table with a `skid_depths` sub-table.

Hex strings such as `"0x10000000"` are accepted for `base_addr` and `addr_range`.

16.2 Connectivity CSV

The connectivity matrix is a CSV with the first column labeled `master\slave` and subsequent columns named after slaves. Example:

```
master\slave,ddr,sram,apb_periph
cpu,1,1,0
dma,1,0,1
```

If `connectivity_file` is `None`, `find_connectivity_csv(ports_file)` searches for `<stem>_connectivity.csv` next to the TOML file.

The loader builds a dictionary `connectivity[master_name] = set(slave_names)`.

16.3 `BridgeConfig` construction

`load_config()` returns a `BridgeConfig` dataclass (`bridge_pkg/config.py`) populated with:

- `masters`: list of `PortSpec` (direction="master")
- `slaves`: list of `PortSpec` (direction="slave")
- `connectivity`: dict mapping master names to sets of slave names
- bridge-level fields such as `variants`, `xbar_pipeline`, `arbitration`, `mon_preset`, `use_cfg_regblock`, `mon_group`

The subtractive catch-all slave is **not** added by the loader; `BridgeModuleGenerator` appends it internally during emission.

16.4 Normalization

- Port prefixes are **not** normalized by the loader; normalization happens later in `bridge_generator.py` via `_normalize_prefix()`.
- `MonGroupConfig` normalizes the deprecated `"axil"` spelling to `"axil4"` in `__post_init__`.
- `dest_width` and `id_width` are not relevant to Bridge; those are Delta-specific.

17 Validation

Validation runs immediately after loading. The function `validate_config(masters, slaves, connectivity)` in `bridge_pkg/config_validator.py` is the single gatekeeper.

17.1 Validation order

1. **Per-port required fields** — `validate_required_fields(port)` for every master and slave.
2. **Channel and protocol values** — `validate_channels()` and `validate_protocol()`.
3. **Protocol constraints** — APB, WB4, AXI3, CDC.
4. **AXI5 feature validation** — `validate_axi5()`, wide-feature checks, poison connectivity, atomic read-return rules, axil5 feature checks.
5. **Slave ID width** — `validate_slave_id_widths()`.
6. **Address map** — `validate_address_map()`.
7. **Master-slave compatibility** — `validate_master_slave_compatibility()`.
8. **Dropped-sideband warnings** — `warn_axi5_dropped_sideband()` (non-fatal).

17.2 Error behavior

Every validator raises `ValidationError(message)` on failure. `bridge_generator.py` catches the exception, prints the traceback, and returns `(False, [])` so the bulk driver can count the failure.

17.3 Examples of validation failures

Scenario	Validator	Message highlights
`channels = "rx"`	`validate_channels`	`Invalid channels 'rx'... Must be one of {'rw', 'rd', 'wr'}`
`protocol = "axi6"`	`validate_protocol`	`Invalid protocol 'axi6'... Must be one of {'axi3', 'axi4', 'axi5', 'apb', 'apb5', 'axil', 'axil5', 'wb4'}`
APB with `id_width = 4`	`validate_apb_constraints`	`must have id_width=0`
AXI5 `poison` on a path to an AXI4 slave	`validate_axi5_poison_connectivity`	`connected slave cannot carry it natively`
Overlapping slave windows	`validate_address_map`	`Address overlap detected`
`wr` master to `rd` slave	`validate_master_slave_compatibility`	`cannot connect to read-only slave`

17.4 Adding a new validation rule

To add a rule:

1. Write a validator function that accepts `(masters, slaves, connectivity)` or a single `PortSpec`.
2. Call it from `validate_config()` at the appropriate point in the order above.
3. Update this chapter and the configuration chapter with the new rule.

Keep validators stateless and raise `ValidationError` only; do not mutate the config.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

18 Adapter and Crossbar Emission

After validation, `bridge_generator.py` calls `_emit_bridge_variant()` for each variant in `BridgeConfig.variants`. That function translates `PortSpec` objects into `MasterConfig`/`SlaveInfo` objects and drives `BridgeModuleGenerator`.

18.1 Translation to emitter structs

`MasterConfig` and `SlaveInfo` are lightweight structs consumed by the generators. Key fields:

Field	`MasterConfig`	`SlaveInfo`
`name`	port name	port name
`prefix`	normalized prefix	normalized prefix
`data_width`	int	int
`addr_width`	int	int
`id_width`	int	int
`channels`	`rw`/`wr`/`rd`	`rw`/`wr`/`rd`
`slave_connections`	list of slave indices	—
`protocol`	string	string
`use_monitor`	bool	bool
`mon_enables`	dict	dict
`axi5_features`	list	list
`cdc`	—	bool
`enable_ooo`	—	bool
`internal`	—	bool

18.2 `BridgeModuleGenerator` construction

`_emit_bridge_variant()` instantiates `BridgeModuleGenerator` with:

```
BridgeModuleGenerator(
    bridge_name=output_name,
    enable_monitoring=use_monitor,
    internal_axil_group=internal_axil_group,
    use_all_monitors=...,
    use_no_monitors=...,
    use_cfg_regblock=...,
    mon_group=...,
    xbar_pipeline=...,
    arbitration=...,
    qos_aging_shift=...,
)
```

Then it calls `gen.add_master(master)` and `gen.add_slave(slave)` for every port, and finally `gen.generate_all(bridge_dir)`.

18.3 Sub-generators

`BridgeModuleGenerator.generate_all()` returns a dict mapping a tag to a file path. The tags are:

Tag	Generator class	Output file
`package`	`PackageGenerator`	`<bridge>_pkg.sv`
`adapter_<name>`	`AdapterGenerator`	`<name>_adapter.sv`
`slave_adapter_<name>`	`SlaveAdapterGenerator`	`<name>_adapter.sv`
`bridge`	`BridgeModuleGenerator` itself	`<bridge>.sv`
`crossbar`	`CrossbarGenerator`	`<bridge>_xbar.sv`

The package is always emitted first so later modules can `import` it. Adapter file names are based on the port `name`, not the prefix.

18.4 Master adapter (`AdapterGenerator`)

The master adapter:

1. Declares the external port with the configured prefix and channel subset.
2. Optionally instantiates a timing wrapper (`axi4_slave_wr/rd`, `_monlite` variants, or AXI5 boundary wrappers).
3. Optionally instantiates a protocol converter (`apb4_to_axi4`, `axi3_to_axi4`, `wb4_to_axi4`, `axi5_atomic_filter`, etc.).
4. Generates per-width output paths (32b, 64b, 128b, 256b, 512b) and inserts `axi4_dwidth_converter_{wr,rd}` or `axil_to_axi4_wide_align_{wr,rd}` only where needed.
5. Emits address decode logic producing `<name>_slave_select_aw` / `ar` and a `bridge_id` sideband.

18.5 Slave adapter (`SlaveAdapterGenerator`)

The slave adapter:

1. Declares the external slave port with the configured prefix and protocol.
2. Optionally instantiates a timing wrapper or protocol shim (`axi4_to_apb4_shim`, `axi4_to_axi3`, `axi4_to_wb4`, AXI5 boundary wrappers).
3. Carries the in-order `bridge_id` FIFO for response routing.
4. Handles CDC if `cdc=true` by inserting `axi4_cdc_{wr,rd}`.

18.6 Crossbar (`CrossbarGenerator`)

The crossbar module wires all master-adapter outputs to all slave-adapter inputs. It contains:

- AW and AR request matrices.
- Per-slave round-robin arbiters (or QoS arbiters when `arbitration="qos"`).
- W-owner FIFOs and response routing by `bridge_id`.
- Optional registered crossbar stage when `xbar_pipeline=true`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

19 Filelist and Test Generation

After RTL emission, `_emit_bridge_variant()` writes the filelist and optionally generates cocotb tests.

19.1 Filelist generation

The filelist is written to `rtl/filelists/<bridge_name>.f`, two levels above the bridge directory. It is a complete compile closure using `$REPO_ROOT`-relative paths.

19.1.1 Filelist sections

1. **Include directory** — `+incdir+$REPO_ROOT/rtl/amba/includes`.
2. **Reset macro header** — `-f $REPO_ROOT/rtl/amba/filelists/reset_defs.f`.
3. **Monitor packages** — `-f $REPO_ROOT/rtl/amba/filelists/monitor_pkgs.f` (always included, even for `"no"` variants, because the subtractive catch-all references them).
4. **Subtractive catch-all** — `-f $REPO_ROOT/rtl/amba/filelists/axi4_subtractive_slave.f`.
5. **Generated files** — package first, then other generated files sorted by tag.
6. **AXI4/AXI5 wrapper closures** — `axi4_slave_wr.f`, `axi4_slave_rd.f`, `axi4_master_wr.f`, `axi4_master_rd.f`, plus AXI5 variants when needed.
7. **Converter closures** — `axi_data_upsize.f`, `axi_data_dnsize.f`, `axi4_dwidth_converter_{wr,rd}.f`, `axil_to_axi4_wide_align_{wr,rd}.f`.
8. **Protocol shim closures** — APB, AXI-Lite, AXI3, Wishbone, AXI5 atomic filters/read-return trackers as needed.
9. **CDC closures** — `axi4_cdc_wr.f`, `axi4_cdc_rd.f` if any slave has `cdc=true`.
10. **Monitor infrastructure** — `_monlite` wrapper filelists and `monbus_arbiter.f`, plus the selected `monbus_<p1>_<p2>_group.f` for `"mon"` variants.
11. **Verilator waiver** — `bridge_regblock.vlt` when a cfg regblock is present.

The filelist is generated by string construction in `bridge_generator.py`, not by a template. Every dependency is pulled in via the owning component's closure filelist rather than by hand-listing individual `.sv` files.

19.2 Test generation

When `--generate-tests` is passed:

- For `"no"` variants, `generate_tests()` emits a TB class in `dv/tbclasses/` and a test runner in `dv/tests/`.
- For `"mon"` variants, `generate_monitor_tests()` emits a TB class and a monitor-stress test runner.

Both functions use Jinja2 templates from `bin/bridge_pkg/jinja_templates/`:

Template	Output
`bridge_tb_class.py.j2`	`dv/tbclasses/<class_module>.py`
`bridge_test_file.py.j2`	`dv/tests/test_<bridge>.py`
`bridge_monitor_test.py.j2`	`dv/tests/test_<bridge>_monitor.py`

The TB class name is derived from the bridge name by `build_tb_class_name()`. The import block is selected by `_tb_import_block()`; if the output path is not dotted-importable (for example, because of the hyphen in `fabric-gen-ip`), the generated test falls back to a `sys.path` insertion.

---

20 Directory Layout

For each variant the generator creates a directory under the requested output dir:

```
rtl/generated/<bridge_name>/
├── <bridge_name>_pkg.sv
├── <bridge_name>.sv
├── <bridge_name>_xbar.sv
├── <master0>_adapter.sv
├── <master1>_adapter.sv
├── ...
├── <slave0>_adapter.sv
├── <slave1>_adapter.sv
├── ...
├── <bridge_name>.toml              (copied source config)
├── <bridge_name>_connectivity.csv  (copied connectivity matrix)
├── <bridge_name>_cfg_pkg.sv        (when use_cfg_regblock=true)
└── <bridge_name>_cfg.sv            (when use_cfg_regblock=true)

rtl/filelists/
└── <bridge_name>.f
```

If `--generate-tests` is used, the following are also written:

```
dv/tbclasses/
└── <tb_class_module>.py
dv/tests/
└── test_<bridge_name>.py
```

For "mon" variants the test runner is named `test_<bridge_name>_monitor.py`.

20.1 Variant directories

The "no" variant writes to <base_name>/. The "mon" variant writes to <base_name>_mon/.

Example for a config with name = "bridge_mix_e" and variants = ["no"]:

```
rtl/generated/bridge_mix_e/
├── bridge_mix_e_pkg.sv
├── bridge_mix_e.sv
├── bridge_mix_e_xbar.sv
├── cpu_axi3_adapter.sv
├── dma_axi4_adapter.sv
├── host_apb_adapter.sv
├── ddr_axi3_adapter.sv
├── sram_axi4_adapter.sv
├── apb_periph_adapter.sv
├── subtractive_adapter.sv
├── bridge_mix_e.toml
└── bridge_mix_e_connectivity.csv
```


---

21 Package File

The package file (`<bridge_name>_pkg.sv`) is generated by `PackageGenerator` and is always the first file compiled.

21.1 What it contains

- `BRIDGE_ID_WIDTH` — width of the internal `bridge_id` sideband.
- `NUM_MASTERS` / `NUM_SLAVES` — port counts.
- Localparam constants derived from the topology.
- Typedefs for the internal AXI4 structs used by the crossbar and adapters, such as `axi4_aw_t`, `axi4_w_<width>_t`, `axi4_b_t`, `axi4_ar_t`, `axi4_r_<width>_t`.

The exact set of typedefs depends on the unique data widths present in the bridge. For example, `bridge_mix_e` uses 32-bit and 64-bit paths, so its package defines `axi4_w_32b_t`, `axi4_r_32b_t`, `axi4_w_64b_t`, and `axi4_r_64b_t`.

21.2 Excerpt from a generated package

```
package bridge_mix_e_pkg;

    localparam int NUM_MASTERS = 3;
    localparam int NUM_SLAVES  = 4;  // includes subtractive catch-all
    localparam int BRIDGE_ID_WIDTH = 2;

    typedef struct packed {
        logic [3:0]  id;
        logic [31:0] addr;
        logic [7:0]  len;
        logic [2:0]  size;
        logic [1:0]  burst;
        logic        lock;
        logic [3:0]  cache;
        logic [2:0]  prot;
        logic [3:0]  qos;
        logic [3:0]  region;
        logic        user;
    } axi4_aw_t;

    // ... additional structs for W, B, AR, R at each used width ...

endpackage : bridge_mix_e_pkg
```

21.3 Why it matters

Because the top module imports this package, any change to widths or counts changes the package and therefore every generated file in the same variant. This is one reason partial regeneration is unsafe.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

22 Top Module

The top module (`<bridge_name>.sv`) is the integration shell. It instantiates the subtractive catch-all, every master adapter, the crossbar, and every slave adapter.

22.1 Module signature

The top module takes **no parameters**. All widths are fixed at generation time and come from the package:

```
module bridge_mix_e
    import bridge_mix_e_pkg::*;
(
    input  logic aclk,
    input  logic aresetn,

    // Master 0: cpu_axi3 (rw)
    input  logic [3:0] cpu_axi3_awid,
    input  logic [31:0] cpu_axi3_awaddr,
    // ... all AXI3/AXI4/APB/Wishbone channels per port protocol ...

    // Slave 0: ddr_axi3
    output logic [5:0] ddr_axi3_awid,
    // ...

    // Unmapped-address status (subtractive slave)
    output logic        unmapped_irq,
    output logic [31:0] unmapped_addr,
    output logic [7:0]  unmapped_count,
    input  logic        unmapped_clear
);
```

The exact port list is derived from the port protocol, channel subset, and widths.

22.2 Internal structure

The top module contains:

1. **`axi4_subtractive_slave`** — always present, terminates unmapped addresses with DECERR and exposes the `unmapped_*` status pins.
2. **Master adapter instances** — one per master, named `<name>_adapter`.
3. **Crossbar instance** — `<bridge_name>_xbar`, wired to all adapter outputs.
4. **Slave adapter instances** — one per external slave plus one `subtractive_adapter` for the catch-all.

22.3 Excerpt: subtractive catch-all instantiation

```
axi4_subtractive_slave #(
    .AXI_ID_WIDTH   (6),
    .AXI_ADDR_WIDTH (32),
    .AXI_DATA_WIDTH (64),
    .AXI_USER_WIDTH (1),
    .UNIT_ID        (8'd3),
    .AGENT_ID       (16'h5B00)
) u_subtractive (
    .aclk          (aclk),
    .aresetn       (aresetn),
    // ... AXI channels ...
    .o_hit_irq     (unmapped_irq),
    .o_hit_addr    (subtractive_hit_addr),
    .o_hit_count   (unmapped_count),
    .i_hit_clear   (subtractive_hit_clear)
);
```

22.4 Excerpt: adapter wiring pattern

```
cpu_axi3_adapter u_cpu_axi3_adapter (
    .aclk(aclk),
    .aresetn(aresetn),
    .cpu_axi3_awid(cpu_axi3_awid),
    // ... external AXI3 pins ...
    .slave_select_aw(cpu_axi3_slave_select_aw),
    .bridge_id_aw(cpu_axi3_bridge_id_aw),
    .slave_select_ar(cpu_axi3_slave_select_ar),
    .bridge_id_ar(cpu_axi3_bridge_id_ar),
    // ... width-specific internal paths ...
);
```

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

23 Adapters and Crossbar

23.1 Master adapter (<name>_adapter.sv)

The master adapter sits between an external master and the internal crossbar. Its responsibilities are:

- Expose the external port with the configured prefix and protocol.
- Insert timing-isolation wrappers when requested.
- Insert protocol converters for non-AXI4 master protocols (`apb`, `apb5`, `axil`, `axil5`, `wb4`, `axi3`, `axi5`).
- Generate per-width internal paths and instantiate width converters only where master and slave widths differ.
- Decode addresses and emit `slave_select_aw/ar` plus `bridge_id_aw/ar`.

23.1.1 Per-width paths

For each unique slave width the master connects to, the adapter emits a separate internal AXI4 path. If the master width matches the slave width, the path is direct; otherwise it uses a converter. Example from `bridge_mix_e`:

```
// cpu_axi3 Adapter outputs
logic [NUM_SLAVES-1:0] cpu_axi3_slave_select_aw;
logic [BRIDGE_ID_WIDTH-1:0] cpu_axi3_bridge_id_aw;
// 32b path
axi4_aw_t     cpu_axi3_32b_aw;
axi4_w_32b_t  cpu_axi3_32b_w;
// 64b path
axi4_aw_t     cpu_axi3_64b_aw;
axi4_w_64b_t  cpu_axi3_64b_w;
```

23.2 Slave adapter (<name>_adapter.sv)

The slave adapter sits between the crossbar and an external slave. Its responsibilities are:

- Convert the internal AXI4 face to the external protocol.
- Carry the in-order `bridge_id` FIFO for response routing.
- Handle CDC if `cdc=true`.
- Expose APB, AXI-Lite, AXI3, AXI5, or Wishbone pins as configured.

23.2.1 Response routing

Each slave adapter keeps an in-order FIFO of `bridge_id` values pushed on the address handshake and popped on the response. The returned AXI ID is **not** used for routing; the FIFO head determines which master receives the response.

23.3 Crossbar (<bridge_name>_xbar.sv)

The crossbar module:

- Accepts per-master decode outputs (`slave_select_*`, `bridge_id_*`).
- Builds AW/AR request matrices.
- Instantiates per-slave round-robin arbiters with grant locked to the address handshake.
- Routes W beats via a per-slave W-owner FIFO.
- Muxes responses back to masters using the `bridge_id` sideband.
- Optionally inserts a registered stage when `xbar_pipeline=true`.

The crossbar is protocol-agnostic internally; all protocol conversion happens in the adapters.

24 Filelist

The filelist (`<bridge_name>.f`) declares the complete compile closure for one variant. It is generated in `bridge_generator.py` by `_emit_bridge_variant()`.

24.1 Path convention

All paths inside the filelist are anchored to `$REPO_ROOT`. This allows the filelist to be included from another filelist regardless of the including file's base directory.

24.2 Example filelist head

```
# Include directories
+incdir+$REPO_ROOT/rtl/amba/includes

# Reset macro header (`ALWAYS_FF_RST / `RST_ASSERTED)
-f $REPO_ROOT/rtl/amba/filelists/reset_defs.f

# Monitor packages (must precede any module that references them)
-f $REPO_ROOT/rtl/amba/filelists/monitor_pkgs.f

# Subtractive catch-all slave (unmapped-address terminator)
-f $REPO_ROOT/rtl/amba/filelists/axi4_subtractive_slave.f

# Bridge RTL files (generated)
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/bridge_mix_e_pkg.sv
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/bridge_mix_e_xbar.sv
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/bridge_mix_e.sv
$REPO_ROOT/projects/components/fabric-gen-ip/bridge/rtl/generated/
bridge_mix_e/apb_periph_adapter.sv
...

# AXI4 Wrapper modules (timing isolation)
-f $REPO_ROOT/rtl/amba/filelists/axi4_slave_wr.f
-f $REPO_ROOT/rtl/amba/filelists/axi4_slave_rd.f
-f $REPO_ROOT/rtl/amba/filelists/axi4_master_wr.f
-f $REPO_ROOT/rtl/amba/filelists/axi4_master_rd.f

# Width converters
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi_data_upsize.f
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi_data_dnsize.f
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi4_dwidth_converter_rd.f
-f
$REPO_ROOT/projects/components/utility-ip/converters/rtl/filelists/
axi4_dwidth_converter_wr.f
```

24.3 Filelist is generated, not hand-maintained

Because the converter and wrapper selections depend on the port protocols, the filelist is regenerated every time the bridge is regenerated. Do not hand-edit a generated filelist; instead, change the generator's selection logic and regenerate.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

25 Generated Tests

When `--generate-tests` is passed, the generator emits a cocotb testbench class and a pytest runner for each variant.

25.1 TB class

The TB class is written to `dv/tbclasses/<tb_class_module>.py`. Its name is derived from the bridge name:

```
bridge_2x2_rw  → Bridge2x2RwTB
bridge_mix_e   → BridgeMixETB
```

The class inherits from `TBBase` and implements:

- `setup_clocks_and_reset()` — drives `aclk` and `aresetn`.
- `assert_reset()` — drives `self.dut.aresetn.value = 0`.
- `deassert_reset()` — releases reset.

It constructs `GAXIMaster` / `GAXISlave` BFMs per port using the AXI4 field configs derived from the generated widths.

25.2 Test runner

The test runner (`dv/tests/test_<bridge>.py`) contains `cocotb.test` functions and pytest wrappers. Each wrapper names its `testcase=` explicitly. Example shape:

```
@cocotb.test()
async def basic_read_write(dut):
    tb = Bridge2x2RwTB(dut)
    await tb.run_basic_read_write()

@pytest.mark.parametrize("seed", [0, 1, 2])
def test_bridge_2x2_rw_basic(seed):
    run_test(
        module="test_bridge_2x2_rw",
        testcase="basic_read_write",
        extra_env={"SEED": str(seed)}
    )
```

25.3 Monitor stress test

For "mon" variants, generate_monitor_tests() emits:

- A TB class with the full monbus/cfg surface.
- test_<bridge>_monitor.py driving backpressure and error injection phases scaled by TEST_LEVEL.

25.4 Regeneration discipline for tests

Tests are generated artifacts. After any change to the test templates or to widths that affect BFM construction, regenerate tests with `--generate-tests` from a clean state. Hand-written tests for generated DUTs should live in separate files (for example, `test_bridge_2x2_rw_tracking.py`) and not be overwritten by the generator.

---

26 Full Regeneration Rule

Generated RTL must be treated like compiler output: when the compiler changes, rebuild everything.

26.1 Why partial regeneration is dangerous

- The package file defines widths used by every other generated file.
- Converter selection logic affects adapter port lists and the filelist.
- The `bridge_id` width is global; a change changes the crossbar and every adapter.
- The filelist pulls in external IP closures; an interface change there must be reflected in every generated bridge.

26.2 Required workflow

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean
make all
```

If tests are generated:

```
python bridge_generator.py --bulk bridge_batch.csv --generate-tests
cd ../dv/tests
make clean-all && make run-all-func
```

26.3 Code changes that trigger full regeneration

- `bin/bridge_generator.py`
- Any file under `bin/bridge_pkg/`
- Any Jinja template under `bin/bridge_pkg/jinja_templates/`
- Any shared converter/wrapper IP whose interface changed

26.4 Files that are never regenerated

- `rtl/bridge_cam.sv` — hand-written CAM, unused by every generated bridge.
- Anything outside `rtl/generated/` and `rtl/filelists/`.

26.5 Version-mismatch symptoms

- Tests that previously passed now fail.
- Simulation errors about missing signals or mismatched widths.
- Address decode routing to wrong slaves.
- Verilator warnings about undeclared identifiers in the package.

If any of these appear after a generator change, the first recovery step is `make clean && make all`.

---

27 Generated Examples

The checked-in `rtl/generated/` directory contains a variety of fixtures that serve as regression anchors and integration examples.

27.1 Representative fixtures

Fixture	Masters	Slaves	Notable features
`bridge_2x2_rw`	2	2	Plain AXI4, 32-bit addresses, per-port widths.
`bridge_2x2_axi5`	2	2	Native AXI5 fabric with MTE/chunking.
`bridge_1x2_rw_apb5`	1	2	APB5 slave, single master.
`bridge_2x2_axi3_masters`	2	2	AXI3 masters on AXI4 fabric.
`bridge_mix_e`	3	3	AXI3 + AXI4 + APB masters → AXI3 + AXI4 + APB slaves.
`bridge_1x2_rd_regblock_mon`	1	2	`use_cfg_regblock=true`, `"mon"` variant.
`bridge_2x2_rw_qos_pipe`	2	2	`arbitration="qos"`, `xbar_pipeline=true`.

27.2 How to verify a regeneration

After `make all`, every fixture in `rtl/generated/` should be byte-identical to the checked-in version **if** the generator and inputs have not changed. Because the generated files carry a generation-date-free header, a clean regeneration should not introduce spurious diffs.

To compare:

```
cd projects/components/fabric-gen-ip/bridge
# Regenerate everything
make -C bin clean && make -C bin all
# Check for unexpected diffs
git diff -- rtl/generated/ | head -n 200
```

(Note: the user instruction forbids running git commands; the command above is for illustration only and is the responsibility of the agent that owns the shared tree.)

If the generator code changed intentionally, the diffs are expected and must be reviewed.

27.3 Fixture as documentation

The `bridge_mix_e` fixture is the canonical mixed-protocol example. Its TOML and generated top module are referenced throughout this specification. To understand how a new protocol or width combination is emitted, generate a minimal fixture and compare it to the closest existing one.

28 DV Suite

Bridge uses a cocotb/pytest DV suite in `dv/tests/`. Test classes live in `dv/tbclasses/`; runners live in `dv/tests/`.

28.1 Test structure

```
dv/
├── tbclasses/
│   ├── bridge2x2_rw_tb.py          # per-config TB class
│   ├── bridge_mix_e_tb.py
│   └── ...
└── tests/
    ├── conftest.py
    ├── test_bridge_2x2_rw.py       # generated
    ├── test_bridge_mix_e.py        # generated
    ├── test_bridge_2x2_rw_tracking.py   # hand-written
    ├── test_bridge_1x2_rw_axi5a_atomics.py
    └── ...
```

28.2 Running tests

Use the area Makefile, not bare pytest:

```
cd projects/components/fabric-gen-ip/bridge/dv/tests

# Specific test, gate level
make run-bridge_2x2_rw-gate

# All functional tests
make clean-all && make run-all-func

# Full regression
make clean-all && make run-all-full
```

Bridge tests run serially because each test consumes approximately 1 GB of memory.

28.3 Test levels

Each generated wrapper exports `REG_LEVEL` cells:

Level	Cells	Description
GATE	72	Smoke / gate-level depth
FUNC	144	Functional depth
FULL	216	Stress depth

The TB reads the depth profile from `dv/tbclasses/bridge_levels.py`.

28.4 Verification coverage

The DV suite covers:

- Basic read/write connectivity per master-slave pair.
- Boundary probe (address decode miss and subtractive catch-all).
- Arbitration fairness and multi-master contention.
- Burst transfers and packet atomicity.
- Width conversion paths.
- APB/AXI-Lite/AXI3/Wishbone slave conversions.
- AXI5 sideband propagation and atomics.
- Monitor stress (`*_mon_monitor.py`) and cfg regblock programming.

28.5 When generator changes affect DV

After any generator change that affects the generated port list, width, or filelist, regenerate the tests and run `make clean-all && make run-all-func` before declaring the change safe.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

29 Lint and Synthesis Checks

Every generated bridge is expected to pass Verilator lint and out-of-context synthesis.

29.1 Verilator lint

The `rtl/Makefile` runs lint:

```
cd projects/components/fabric-gen-ip/bridge/rtl
make verilator
```

This lints every generated fixture using its filelist. The generator aims for lint-clean output; any new warning class must be either fixed in the generator or documented with a scoped waiver.

29.2 Out-of-context synthesis

The FPGA flow in `projects/components/fabric-gen-ip/bridge/fpga/` supports out-of-context synthesis:

```
cd projects/components/fabric-gen-ip/bridge/fpga
make synth BRIDGE=bridge_2x2_rw PART=xc7a100tcsg324-1 CLK_NS=10
```

Resource and timing tables in the HAS/MAS are produced by this flow.

29.3 When to run checks

Run lint and at least one synthesis check after:

- Adding a new protocol or option.
- Changing the adapter or crossbar emission templates.
- Updating converter/wrapper dependencies.

A generator change that passes simulation but fails synthesis is not done.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

30 Adding a Protocol

Adding a new protocol to Bridge means adding a new value to the `protocol` enum, validating it, selecting the right converter/wrapper in the adapters, and updating the filelist logic.

30.1 Step 1: Add the protocol to the validator

In `bridge_pkg/config_validator.py`:

1. Add the protocol string to `valid_protocols` in `validate_protocol()`.
2. Add a new `validate_<proto>_constraints()` function if the protocol has special rules (for example, allowed widths, channel restrictions, ID width rules).
3. Call the new validator from `validate_config()`.

30.2 Step 2: Add converter/wrapper selection

In `bridge_pkg/generators/adapter_generator.py`:

- For a master protocol, decide which front-end converter module to instantiate.
- Add the protocol to the width-converter path selection.
- Emit the correct external port signals.

In `bridge_pkg/generators/slave_adapter_generator.py`:

- For a slave protocol, decide which completer shim to instantiate.
- Wire the internal AXI4 face to the shim.

30.3 Step 3: Update the filelist

In `bridge_generator.py` `_emit_bridge_variant()`:

- Add `if any(m.protocol.lower() == '<proto>' for m in config.masters):` block to pull in the master-side converter closure.
- Add the corresponding slave-side block.

Always `-f` the converter's own closure filelist; do not hand-list individual `.sv` files.

30.4 Step 4: Add a representative fixture

Add a TOML/CSV pair to `bin/test_configs/` and a line to `bridge_batch.csv` so the new protocol is exercised in regression.

30.5 Step 5: Run the full regeneration and test

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean && make all
python bridge_generator.py --bulk bridge_batch.csv --generate-tests
cd ../dv/tests
make clean-all && make run-<new_fixture>-func
```

30.6 Example: how APB was added

APB master support uses `apb4_to_axi4` / `apb5_to_axi4` front-end converters. APB slave support uses `axi4_to_apb4_shim` / `axi4_to_apb5_shim`. The validator enforces `channels="rw"`, `data_width` in `{8,16,32}`, and `id_width=0`. The filelist pulls in the converter closures conditionally.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

31 Adding a Bridge-Level Option

Bridge-level options live in the `[bridge]` TOML table and become fields of `BridgeConfig`.

31.1 Step 1: Add the dataclass field

In `bridge_pkg/config.py`, add a field to `BridgeConfig` with a sensible default:

```
@dataclass
class BridgeConfig:
    ...
    my_option: bool = False
```

31.2 Step 2: Add the TOML loader mapping

In `bridge_pkg/config_loader.py`, read the new key from the TOML tree and assign it to the `BridgeConfig` instance.

31.3 Step 3: Validate if necessary

If the option has legal-value restrictions or conflicts with other options, add a validator in `bridge_pkg/config_validator.py` and call it from `validate_config()`.

31.4 Step 4: Thread it into the emitter

In `bridge_generator.py` `_emit_bridge_variant()`, pass the option to `BridgeModuleGenerator`:

```
gen = BridgeModuleGenerator(
    ...
    my_option=getattr(config, 'my_option', False),
)
```

31.5 Step 5: Use the option in `BridgeModuleGenerator`

In `bridge_pkg/components/bridge_module_generator.py`:

1. Accept the option in `__init__`.
2. Store it as an instance attribute.
3. Use it to conditionally emit logic, instantiate modules, or modify the port list.

31.6 Step 6: Document and test

- Add the key to this GAS chapter 2.
- Add a fixture to `bridge_batch.csv` that exercises the option.
- Regenerate and run the DV suite.

31.7 Example: xbar_pipeline

xbar_pipeline is a bridge-level option that adds a registered skid stage inside the crossbar. It is loaded from TOML, stored in BridgeConfig, passed to BridgeModuleGenerator, and used by CrossbarGenerator to emit extra pipeline registers.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

32 Adding a Port-Level Option

Port-level options are fields of `PortSpec`. They affect one port at a time.

32.1 Step 1: Add the PortSpec field

In `bridge_pkg/config.py`, add a field to `PortSpec` with a default:

```
@dataclass
class PortSpec:
    ...
    my_port_option: bool = False
```

32.2 Step 2: Load it from TOML

In `bridge_pkg/config_loader.py`, copy the TOML key into the `PortSpec` constructor for both masters and slaves.

32.3 Step 3: Validate it

If the option has constraints, add a validator in `bridge_pkg/config_validator.py`. For example, `cdc` is validated by `validate_cdc_constraints()` to ensure it is only used on AXI4 slave ports.

32.4 Step 4: Thread it into `MasterConfig` / `SlaveInfo`

In `bridge_generator.py` `_emit_bridge_variant()`, pass the option through when building `MasterConfig` or `SlaveInfo`:

```
master_config = MasterConfig(
    ...
    my_port_option=getattr(master_spec, 'my_port_option', False),
)
```

32.5 Step 5: Use it in the adapter generator

In `bridge_pkg/generators/adapter_generator.py` or `slave_adapter_generator.py`, read the option from the config struct and emit conditional logic.

32.6 Step 6: Update the filelist if needed

If the option requires additional IP, add conditional filelist lines in `bridge_generator.py`.

32.7 Example: cdc

The `cdc` port option is a `PortSpec` field. It is validated as AXI4-slave-only, passed through `SlaveInfo`, and used by `SlaveAdapterGenerator` to insert `axi4_cdc_{wr,rd}`. The filelist conditionally pulls in `axi4_cdc_wr.f` and `axi4_cdc_rd.f`.

33 Testing an Extension

Any generator extension must be proven by regeneration, lint, and DV.

33.1 Minimum test plan

1. **Single-bridge generation**

   ```
   python bridge_generator.py --ports test_configs/my_extension.toml
   ```

   Verify the output directory contains the expected files and that the new logic appears in the generated `.sv`.

2. **Validation failure paths**

   Create deliberately invalid TOML files and confirm the generator raises `ValidationError` with a clear message.

3. **Bulk regeneration**

   ```
   make clean && make all
   ```

   Confirm no fixture regresses.

4. **Lint**

   ```
   cd ../rtl && make verilator
   ```

5. **DV smoke**

   ```
   cd ../dv/tests
   make clean-all && make run-<fixture>-func
   ```

6. **Synthesis sanity**

   For area-sensitive options, run out-of-context synthesis on the smallest fixture that exercises the option.

33.2 Regression fixtures

Add at least one fixture to `bridge_batch.csv` that exercises the new protocol or option. If the option is off by default, also add a fixture with the option enabled.

33.3 Documentation

Update this GAS and `CLAUDE.md` if the extension changes the way users configure or extend the generator. Keep the validator rule list, the configuration tables, and the extension-guide steps in sync.